

IMPERIAL

FINAL REPORT

DEPARTMENT OF COMPUTING

IMPERIAL COLLEGE OF SCIENCE, TECHNOLOGY AND MEDICINE

A Multi-Agent LLM Framework for Maintaining Code Quality in Iterative Software Development

Author:
Bernard Moy

Supervisor:
William Knottenbelt

September 9, 2026

Submitted in partial fulfillment of the requirements for the MSc Advanced
Computing of Imperial College London

Abstract

LLM coding agents are increasingly capable of implementing substantial software features from scratch, and are becoming more widely integrated into software development workflows. The adoption of these LLM-based tools has been shown to increase task correctness and substantially boost the development velocity. However, software development is an iterative process, where new features are often extended on top of an existing codebase. Several studies have shown that around successive iterations, LLM-generated code tend to accumulate technical debt and code quality issues more quickly than human-written code. This degradation of code quality can make the codebase become increasingly difficult to maintain over time as it becomes more complex, potentially causing the initial development velocity gain to dissipate after one to two months. While existing approaches have explored frameworks to improve the quality of LLM-generated code through refactoring, there has been a limited focus on mitigating this accumulation of technical debt in an iterative setting where significant features are added over time.

To address this limitation, we first propose a multi-agent LLM-based framework, aimed at preserving code quality throughout iterative software development while maintaining its functional correctness. The framework consists of several specialised agents, namely the *decomposer* for producing a high-level and low-level design plan that define module responsibilities, public interfaces, and their dependency graph, the *analyser* for critically evaluating the proposed design or implementation, and the *coder* for implementing targetted modules according to the proposed design. Since software quality is context-dependent and design decisions often involve trade-offs that require subjective human judgment, our framework also feature human-in-the-loop. The human act as the coordinator by answering agent questions, and reviewing, approving or modifying the code suggestions plan. These agents are prompted using the few-shot chain-of-thought prompting technique, with guided reasoning examples to enhance their performance on complex tasks. To support and guide the agent’s decisions, we also selected and defined a set of design-level and implementation-level code quality metrics.

Second, to support the evaluation of our results, we have compared commonly used LLM-based software engineering benchmarks. We select the SlopCodeBench benchmark for our evaluation, as it supports the evaluation of quality and correctness without revealing the internal design structure. To provide a reference for the code quality metrics from human-written code, we select historical version snapshots from 8 popular GitHub repositories and analyse them for comparison. Furthermore, we create two specialised software engineering problems organised into checkpoints, and provide an expert-designed modular decomposition to enable comparison between designs produced with and without human involvement.

Third, we evaluated our proposed framework to three research questions: (1) How effectively it mitigates the accumulation of code quality issues; (2) How the framework affects functional correctness compared to directly coding; and (3) How human-in-the-loop influences the agent’s design decisions. Our results show that the framework is effective at mitigating the iterative code smells, in particular, it reduces the maximum function cyclomatic complexity by up to $5\times$. The human-in-the-loop evaluation further demonstrates that agents can extract the relevant contextual information and ask targetted questions, aligning the design more closely with human preferences. However, the results also reveal a trade-off between the improvements in code quality and functional correctness, and discovered inconsistencies in model behaviours. These findings motivate future evaluation across a broader variety of models and capability levels, and compute the aggregated results through multiple independent runs to mitigate the variations caused by the non-deterministic behaviour of LLMs.

Acknowledgments

I would like to express my thanks to:

- My supervisor, Professor William Knottenbelt, for his continuous guidance and support given throughout the project. He has continuously encouraged me to explore new research directions, and provided invaluable guidance on resolving unexpected challenges that arose throughout the project.
- My second marker, Dr Tom Crossland, for his detailed feedback on the structure and the language of my interim report. He has also encouraged me to explore alternative approaches, such as identifying popular GitHub repositories for evaluation.
- My parents, for their unconditional love and support throughout my Master's studies.

Keywords

LLM agents, Multi-agent systems, Software quality, Static analysis, Human-in-the-loop

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Objectives	2
1.3	Contributions	4
1.4	Dissertation Roadmap	5
2	Background and Related Works	6
2.1	LLM-based Code Generation	6
2.1.1	LLM-Specialized Techniques	6
2.1.2	Agentic Refactoring	6
2.2	Human-LLM Collaboration	7
2.3	Software Quality	8
2.3.1	Maintainability and Modularity	8
2.3.2	Static Analysis Metrics	9
2.4	Multi-Agent Systems	9
3	Approach	11
3.1	Overview	11
3.2	Decomposer Agent	14
3.3	Analyser Agent	16
3.4	Human Responses	17
3.5	Coder Agent	18
4	Evaluation	21
4.1	Experimental Setup	21
4.1.1	Offline Evaluation	21
4.1.2	Human-in-the-loop Evaluation	24
4.2	Evaluation Results	25
4.2.1	RQ1: How Effective Is Our Approach in Mitigating the Accumulation of Technical Debt in an Iterative Software Development Process?	25
4.2.2	RQ2: How Does Our Multi-Agent Framework Affect the Functional Correctness of LLM-Generated Code When Compared to Direct Coding?	34

4.2.3	RQ3: How Does the Involvement of Human-in-the-Loop Affect the Design Decisions Made by LLMs?	36
4.2.4	Ablation Study	44
4.3	Threats to Validity	46
5	Conclusion and Future Work	48
5.1	Summary of Achievements	48
5.2	Future Work	49
A	Additional UI Images	56
B	Prompts	60
B.1	Decomposer Agent Prompt	60
B.2	Analyser Agent Prompt	62
B.3	Coder Agent Prompt	64
B.4	Test Planner Agent Prompt	65
B.5	Tester Agent Prompt	65
C	Agent Questions to the Human	66
C.1	Problem 1: Iris Recognition CLI Tool	66
C.2	Problem 2: London Underground Arrivals App	68
D	Referenced GitHub Repositories	70

Chapter 1

Introduction

1.1 Motivation

The use of AI coding agents is rapidly becoming increasingly common in the field of software engineering, with 90% of professional developers using them at work at least weekly, and 68% of them using daily as of July 2026 [1]. Tools such as OpenAI Codex [2], Cursor [3] and Claude Code [4] are capable of autonomously planning, writing, and running commands with minimal human intervention, whose usage have risen by 21% and 13% respectively in the first half of 2026, as shown in figure 1.1 [1]. Research has shown that the use of AI agents can increase task correctness by 35%, using only 50% of the user effort measured by the amount of time spent [5], massively boosting the development velocity.

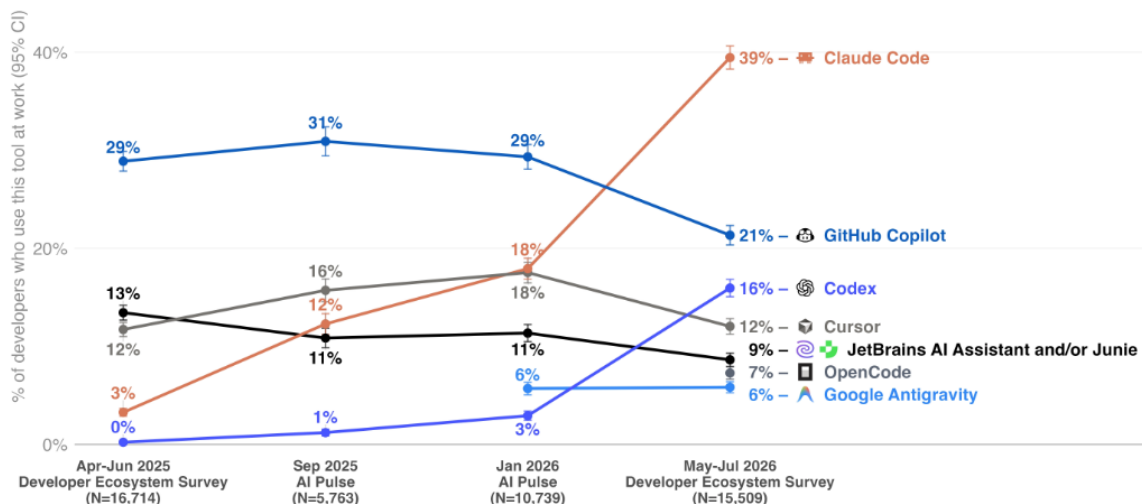


Figure 1.1: Developer Ecosystem Survey, on the AI Tools Used by Developers at Work

However, several studies have raised concerns about the quality and maintainability of LLM-generated code compared to humans. Previous research has shown that the increase in development velocity associated with adopting Cursor, measured by

the number of lines added, concentrated in the initial one to two months [6]. The transient velocity gain is attributed to the accumulation of technical debt, evidenced by the 30% increase in static analysis warnings and 41% increase in code complexity post Cursor adoption, as shown in Figure 1.2. Another study found that LLM agents accumulate complexity and code duplication more quickly than human developers, when extending their own previously written code [7]. Although explicit prompting strategies such as identifying antipatterns to avoid, or requiring the agent to produce a plan before code generation, can improve the initial code quality, these techniques do not prevent the quality degradation over subsequent iterations. [7].

While several approaches [8, 9] have been proposed to address code quality issues in LLM-generated code, these approaches focus on local refactoring after the code has been written, with limited attention to how to prevent the accumulation of technical debt in an iterative development process. In addition, existing work [10] primarily focuses on competitive programming problems. However, while LLMs have been shown to perform well on simple and well-defined coding tasks, they are often outperformed by humans on tasks requiring more complex problem solving, as LLMs tend to over-engineer solutions and produce more complex code [11], as shown in Figure 1.3. Thus, our study specifically focuses on iterative feature development, where substantial functionality is introduced in each checkpoint, and poor architectural decisions or code duplication in earlier stages would accumulate and adversely affect maintainability in later stages.

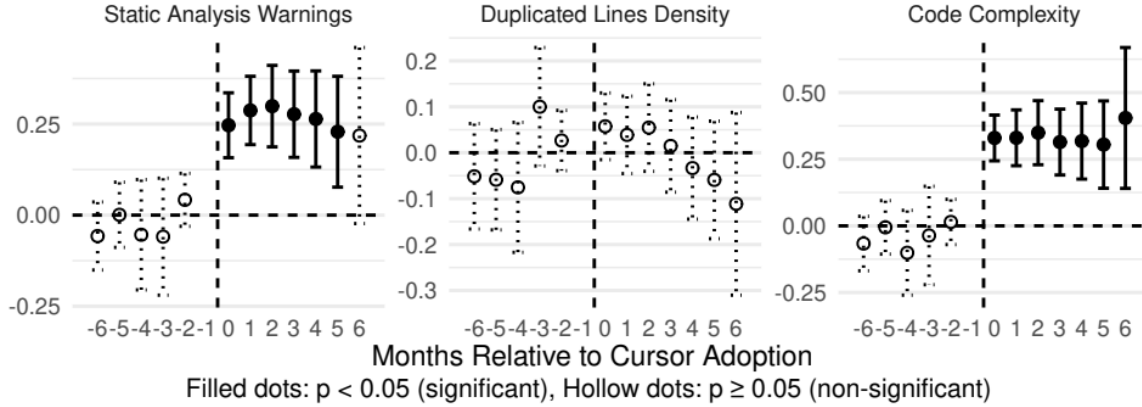


Figure 1.2: The Rise in Static Analysis Warnings, Duplicated Lines and Code Complexity after Cursor Adoption

1.2 Objectives

These findings, together with our identified research gap, highlight the need for agentic systems that continuously assess code quality alongside code generation. To further support architectural decisions that involves ambiguous context or code quality trade-offs, we also introduce human-in-the-loop that enables continuous human involvement in the development process. Therefore, we present the main objective of our research:

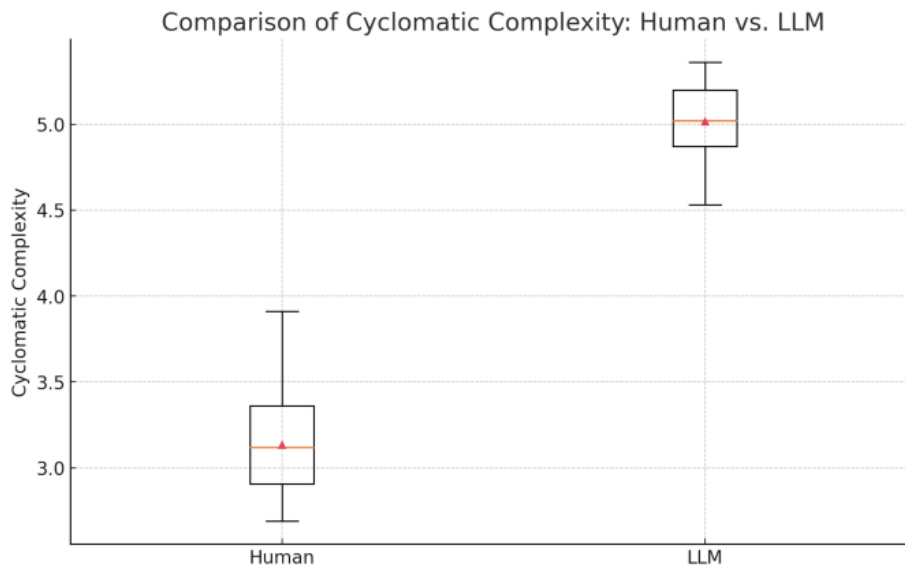


Figure 1.3: LLMs Tend to Produce More Complex Code than Humans

Objective: Design and evaluate a Python-based multi-agent framework featuring human-in-the-loop, in which specialised LLM agents collaborate across software design, analysis, implementation and refactoring. The human acts as the coordinator, providing feedback at appropriate decision points. Each agent is guided by maintainable software engineering principles, with the aim of reducing the quality gap between LLM-generated and human-written code while preserving functional correctness, and mitigating the iterative degradation of code quality as new features are being introduced.

To achieve this objective, the following sub-objectives are defined:

- Design a multi-agent framework that decomposes software development into specialised roles for design, code quality review, and implementation, featuring feedback loops that communicate using shared states.
- Identify and incorporate a set of established and objective software metrics at both the design and implementation levels, to detect potential quality issues and guide refactoring decisions.
- Develop carefully designed prompts using few-shot chain-of-thought prompting, to incorporate human feedback at the appropriate stage of the pipeline, particularly when design trade-offs or missing contextual information requires human judgement.
- Identify and construct suitable benchmark datasets to evaluate code quality and functional correctness throughout iterative software development, and investigate how human-in-the-loop involvement influences the design decisions made by the agents.

To evaluate the effectiveness of our approach in achieving the above objectives, the following research questions are proposed:

RQ1: How effective is our approach in mitigating the accumulation of technical debt in an iterative software development process?

RQ2: How does our multi-agent framework affect the functional correctness of LLM-generated code when compared to direct coding?

RQ3: How does the involvement of human-in-the-loop affect the design decisions made by LLMs?

1.3 Contributions

The contributions of this dissertation project are summarised as follows:

- Design the architecture of a multi-agent LLM framework, that can either run automatically or feature human-in-the-loop. It involves specialised LLMs acting as the decomposer, analyzer, coder and refactor coders that collaborate across two feedback loops, using JSON as an interface for communication. We also identify and compute a list of design and implementation-level metrics, that are generated within the feedback loops. This framework is implemented as a command line tool in Python, with an user interface developed using the Textual UI library (Chapter 3).
- Select the SlopCodeBench benchmark and extract evolving snapshots from a set of popular GitHub repositories (Chapter 4.1.1). We then evaluate the effectiveness of our approach in an automated setting, in preserving code maintainability as software complexity and functionality increases over time. The results demonstrate that the multi-agent approach effectively mitigates iterative technical debt across the readability, simplicity, maintainability, modularity and reusability aspects, leading to code quality that more closely aligns with that of human-written repositories (Chapter 4.2.1).
- Using the SlopCodeBench benchmark, we investigate the trade-off between code quality and functional correctness. To address the observed trade-off, we revise the framework to incorporate test-driven development by introducing the test planner agent and the tester agent. The results reveal that this trade-off persists, while TDD improved the functional correctness, the magnitude of its improvement is model dependent (Chapter 4.2.2).
- We study the effects of featuring human-in-the-loop (HITL) by examining how human involvement influences the design decisions made by the agents. We design two specialised and realistic software projects, and compare the design decisions of the automated and HITL approaches against expert-designed modular decompositions. The results show that agents are capable of asking targeted questions to justify its refactoring decisions, enabling design decisions that more closely aligned with human preferences (Chapter 4.2.3).

- We perform an ablation to assess the contribution of the implementation-level metrics generation component. The results demonstrate that the decomposer–analyser and coder–analyser feedback loops, together with the dependency graph metrics, make substantial independent contributions to improving code quality (Chapter 4.2.4).

1.4 Dissertation Roadmap

The remainder of this dissertation is organised as follows:

- Chapter 2 provides background on LLM-based code generation, human–LLM collaboration, software quality aspects and principles, and existing multi-agent approaches for improving the quality of LLM-generated code.
- Chapter 3 describes the technical architecture and the user interface of the proposed framework. We first provide a high-level overview of the interactions between framework components, followed by a detailed description of the prompt design for each agent, and the specific metrics and contextual information provided to them at each stage.
- Chapter 4 presents the evaluation of the proposed framework. We first introduced the experimental setup, including the criteria of the dataset selection and the set of evaluation metrics used. We then answered our research questions through an automated evaluation on the quality and correctness on problems of the SlopCodeBench dataset, comparing our approach against direct LLM-based coding and human-written repositories. We further investigate the influence of human-in-the-loop on agent design decisions through two specialised problems, and conduct an ablation study by removing the implementation metrics generation component.
- Chapter 5 summarises the contribution of this dissertation project, and outlines future research directions.

Chapter 2

Background and Related Works

2.1 LLM-based Code Generation

2.1.1 LLM-Specialized Techniques

Large Language Models (LLMs), such as Claude Opus 4.5 [12] and GPT-5.2-Codex [13] have demonstrated strong capabilities in coding tasks, achieving remarkable results in software engineering benchmarks such as SWE-bench [14]. Techniques including domain-specific fine-tuning, prompt engineering, and retrieval augmented generation (RAG) have been used to enhance the accuracy and contextual relevance of LLM-generated responses. However, as training or fine-tuning can be expensive, leading providers such as OpenAI have increasingly emphasised prompt-based techniques over fine-tuning [15], with the recent enhancements in LLM context windows.

Prompt engineering, which involves structuring and optimising natural language prompts to guide model behaviour, is widely used to improve LLM performance. Techniques such as chain-of-thought (CoT) prompting that encourage the model to generate intermediate reasoning steps, and few-shot prompting that provides the model with representative examples, have been shown to significantly improve the LLM's ability on complex reasoning tasks [16]. In software development, CoT-based approaches have consistently outperformed direct generation in terms of pass rates across six Python benchmarks [17]. More specialised techniques, such as structured chain-of-thought that describes the reasoning process in terms of program sequence, branch, and loop structures, have demonstrated further improvements in functional correctness, code smells and maintainability [18].

2.1.2 Agentic Refactoring

Refactoring is the process of modifying the structure and design of code without affecting its externally observable behaviour, which is a common practice by AI coding agents and can occur without explicit intent [19]. Examining refactoring behaviour

can therefore help discover the differences in coding patterns between agents and humans. Previous experiments have shown that agents tend to perform fewer high-level refactorings compared to humans and favour localised changes, such as changing variable types and renaming attributes, over broader changes such as adding parameters to system interfaces [19]. Human refactoring patterns have also been found to be more diverse, with a greater emphasis on structural improvements such as moving classes [20]. However, although agents may perform a greater number of refactoring operations than humans, these changes do not necessarily reduce code smells in some cases [20].

While there are plenty of research studying the refactoring ability of LLMs, refactoring is the inherently the opposite of a typical software development workflow, where design comes before coding [21]. In practice, in agent-assisted development, the initial implementation of the software is largely AI driven [5], relying primarily on subsequent refactoring may allow quality issues and antipatterns to accumulate. Failing to incorporate refactoring sprints that keep pace with the fast and high productivity AI-driven workflow, may cause the project to become unmaintainable over time [6]. Therefore, our study aims to incorporate code quality assessment from the design stage, before the codebase becomes unnecessarily complex, tightly coupled and increasingly difficult to refactor.

2.2 Human-LLM Collaboration

Fully-automated LLM-based systems, which lack real-world context, can be insufficient for robust real-world software engineering problems, particularly if they involve ethical consideration or subjective judgment. The optimal result is often obtained through effective human-AI collaboration, allocating tasks across human and AI according to their respective strengths. The RACI framework has been applied to identify software engineering tasks that can be automated using LLM agents, while retaining human involvement in the requirements elicitation and the product roadmap creation tasks, as these tasks reflect the project goals and stakeholder needs. On the other hand, LLMs are involved in automated tasks such as the generation user stories and maintaining product backlogs [22].

Luo et al. proposed CentaurEval, a coding benchmark designed such that the optimal performance requires the collaboration between human reasoning and AI efficiency, while either party working independently achieves suboptimal results [23]. This highlights the potential benefits of human-in-the-loop approaches for coding tasks, particularly for the complex software engineering tasks studied in this project [23]. A key design principle of CentaurEval is *necessary collaboration*, where human excels at higher-order reasoning that outperforms AI in terms of handling contextual judgement for real-world complexity, and AI excels at efficiently handling repetitive and low-level implementation [23].

Human-in-the-loop systems have also been deployed in practical software development workflows. An example is HULA, a framework integrated in Jira, where human

reviews are involved in both the planning and implementation stages [24]. After selecting a software development task, a planner agent first analyses the JIRA issue description to identify relevant files and generate a file-level coding plan, where the human can then directly modify the list of relevant files and edit the plan for each file. Once the plan is approved, the system proceeds to the code generation stage where the human can then review, provide refinements, and request to regenerate the code. By incorporating human feedback to better align code generation with the intent exposed in the issue description, 61% of survey participants agreed that code generated with HULA was easy to read and modify, which helped reduce the time and effort required during initial development [24]. Additionally, participants acknowledged that HULA could promote good documentation practice [24]. These findings demonstrate the potential of human–AI collaboration, in achieving higher functional alignment, improving comprehension and facilitating code maintenance.

2.3 Software Quality

2.3.1 Maintainability and Modularity

Software quality is a broad concept describing the desirable attribute of code, spanning aspects such as functional suitability, security, compatibility and maintainability in the ISO/IEC 25010:2011 Software Quality Model quality model [25]. This project focuses specifically on the structural maintainability of software, particularly on how the code architecture facilitates continued development and feature implementation. Although there are no universal standards, a software with low maintainability usually indicates that it requires a greater effort to understand, modify, extend, and refactor, causing development to be slow and prone to errors.

The concept of “modular programming” dates back to the 1970s [26], describing a programming paradigm that decomposes the codebase into independent modules, each exposing a well-defined public interface for other modules to interact with. Well-designed modules should encapsulate a single and focused responsibility and expose an interface that is easy to use and understand, without revealing implementation details. Dependencies between modules should also be minimised, to limit the extent to which changes propagate across the system as it evolves over time [27].

A core modular design principle is “high cohesion, low coupling”. Cohesion describes how closely related the functionalities are within a single module, and coupling describes how closely do separate modules depend on each other. The integration strength that represents cohesion, and the distance between modules that represents coupling, exhibit a XOR relationship [28]. Functionalities that are closely related to each other should not be scattered far apart, and unrelated functional components should not be encapsulated in the same module. An approach to modular decomposition is the principle of information hiding introduced by DL Parnas, where each module should encapsulate as much complex information as possible, rather than decomposing the system according to the individual steps in its processing pipeline [29].

A well-designed modular system is essential for reducing the long term technical debt and improve maintainability. In tightly coupled systems, where software components depend heavily on each other to function, changes would create a ripple effect on other modules, increasing the risks of unintentional bugs, and introduce difficulty to test in isolation and divide responsibilities across software teams. However, LLM-generated code do not always respect modular boundaries. For example, when they were asked to create a new module in an existing system, they have been observed to violate encapsulation by accessing a private element instead of using the intended public API [30].

2.3.2 Static Analysis Metrics

Tools such as Designite Python [31], Pylint [32], Radon [33] and Sonarqube [34] are static analysis tools that help discover the violation of design principles without code execution, commonly known as “code smells”. These tools are widely used by developers with different years of experience, with “lines of code” and “cyclomatic complexity” being used by 54% and 52% of surveyed developers respectively [35]. Code smells and extreme metric values can serve as indicators for deeper maintainability problems. For example, a function that is too long or feature too many independent control paths, can make it more difficult to understand, extend, modify and test.

Metrics alone, however, are prone to false positives, and are insufficient to capture the full context of the software’s quality. Some metrics such as lack of cohesion of methods (LCOM) are underspecified, having several variants ranging from LCOM1 to LCOM4 that measure cohesion using different definitions. In addition, the majority of existing metrics provide values on an absolute numerical scale, without providing a clear guidance on how these values should be interpreted [35]. One example of this is the maintainability index (MI), combining the halstead volume, cyclomatic complexity and lines of code into a single maintainability score [36], which has been criticised for the use of arbitrary constants and the lack of guidance on how the metric could be improved from reading the numerical score alone [37]. Developers also recognised that the current set of metrics are insufficient to assess the architectural side of quality [35], and smells such as duplicated logic are not easily detectable using traditional approaches. While most existing work uses these tools as the main indicator of software quality, this project uses them as supporting indicators that flag areas to concern, while leveraging the LLM’s reasoning ability to reflect the full picture of different quality aspects.

2.4 Multi-Agent Systems

A multi-agent system is a group of AI agents, with each of them typically assigned a dedicated role. These agents work in a shared environment, either collaboratively or competitively, to handle complex or large scale challenges that would overwhelm a single agent [38]. We base our work on RefAgent [39] and CORE [9], both of which

employ multi-agent architectures to resolve quality issues reported by static analysis tools on LLM-generated code.

RefAgent [39] proposes a multi-agent refactoring workflow. It consists of a context-aware planner agent that invokes `jdeps` to generate a dependency graph, and `DesigniteJava` to generate code quality metrics, that are used to formulate a refactoring plan. The refactoring generator agent would then generate refactored Java code following the instructions. It also features a compiler and tester agent to ensure that functional behaviours are preserved by the refactoring. Evaluation results have shown that LLM agents are able to achieve around 90% unit test pass rate while reducing around 50% of identified code smells, as measured by the QMOOD metric [40]. However, the identification of potential quality issues is mainly provided by the static analysis tool `DesigniteJava`, which inherits some of the limitations previously discussed.

CORE [9] proposes a refactoring workflow with two agents: A proposer LLM that receive quality issues generated by `CodeQL` for Python and `Sonarqube` for Java, used to generate 10 candidate revisions for each file, and a ranker LLM to score the revisions in the order of their likelihood of acceptance. Although the proposer successfully resolves over 80% of the issues, relying on the proposer alone introduced false positives compared to human judgment. The presence of the ranker agent provides an additional review stage by evaluating and surfacing only the top performing candidate revisions, and is able to reduce false positives by 25.8%.

Chapter 3

Approach

3.1 Overview

The primary objective of the project is to develop a multi-agent framework consisting of specialized LLM agents with distinct responsibilities throughout software development. The decomposer is responsible for structuring and writing the design schema, while the analyser would critically evaluate the design or implementation by reading the code smells metrics and giving suggestions. The coder implements the selected modules, and address identified code quality issues.

The framework incorporates two iterative critic loops, between the decomposer and analyser agents for reviewing and improving the software design schema, and between the coder and analyser agents for improving its implementation. They communicate using shared states, where the results written by previous agents, which are commonly structured JSON files, are read by later agents.

As software design often involve subjective and context-dependent design decisions that might be ambiguous and cannot be determined solely from the requirements, the human is continuously involved throughout the analyser agent. When the analyser identifies extra contextual information that it needs in order to justify its suggestions, it would ask human a question for clarification. The human is involved in answering these questions, and can additionally provide feedback and approve the analyser's plan towards the end of each loop. This iterative approach supports the continuous and progressive refinement and discovery of new issues, covering more edge cases and reducing the risk of hallucinations in both the design and implementation level. Figure 3.1 shows the high-level design of the full multi-agent framework with human-in-the-loop implemented in Python, and Figure 3.2 shows the user interface developed with the Textual UI library that the human interacts with.

Each agent is equipped with a specialized prompt, where the few-shot chain of thought prompt engineering technique is applied to encourage more consistent and objective reasoning techniques. The representative examples in the prompt guide

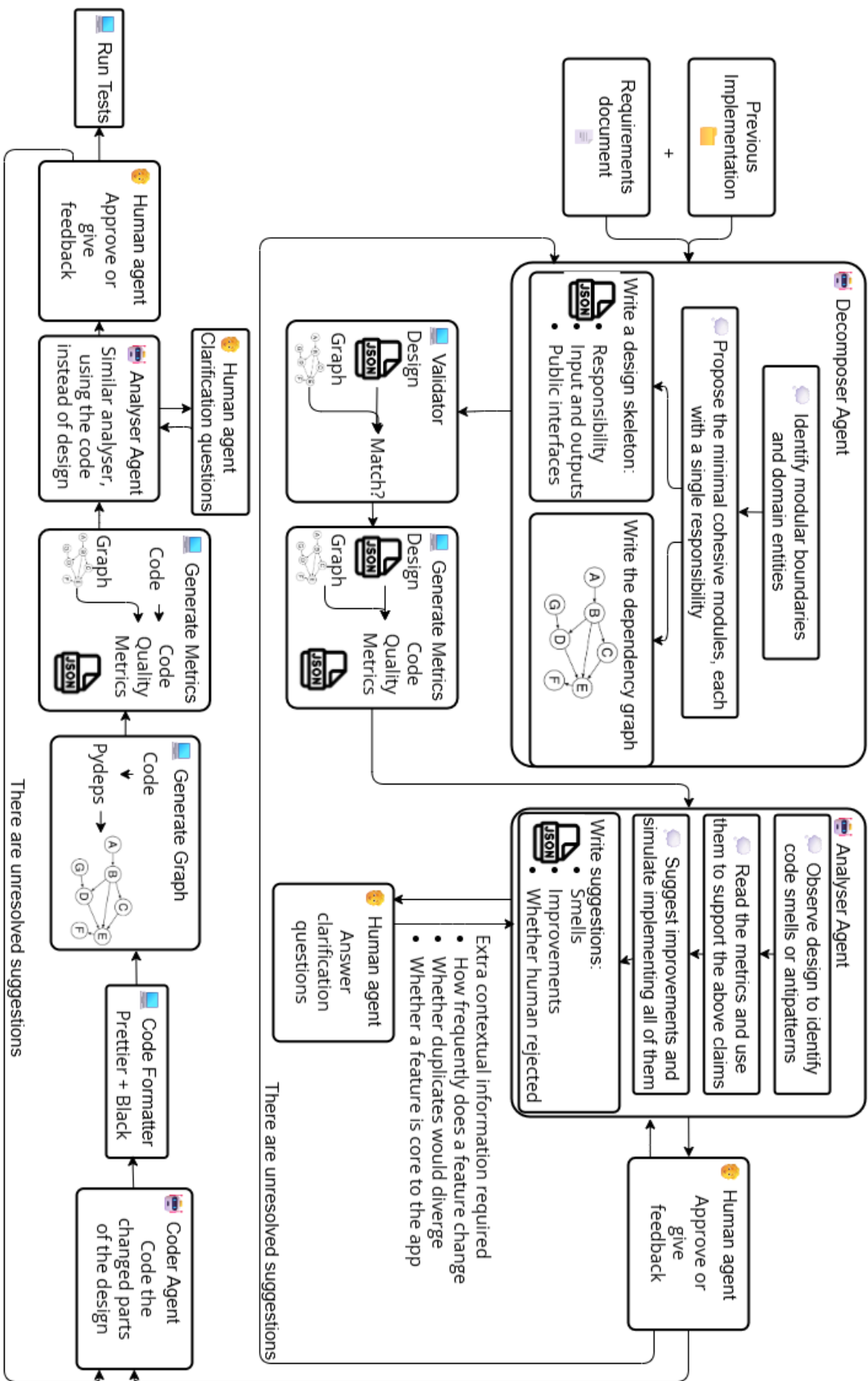


Figure 3.1: High-level design overview of the multi-agent system.

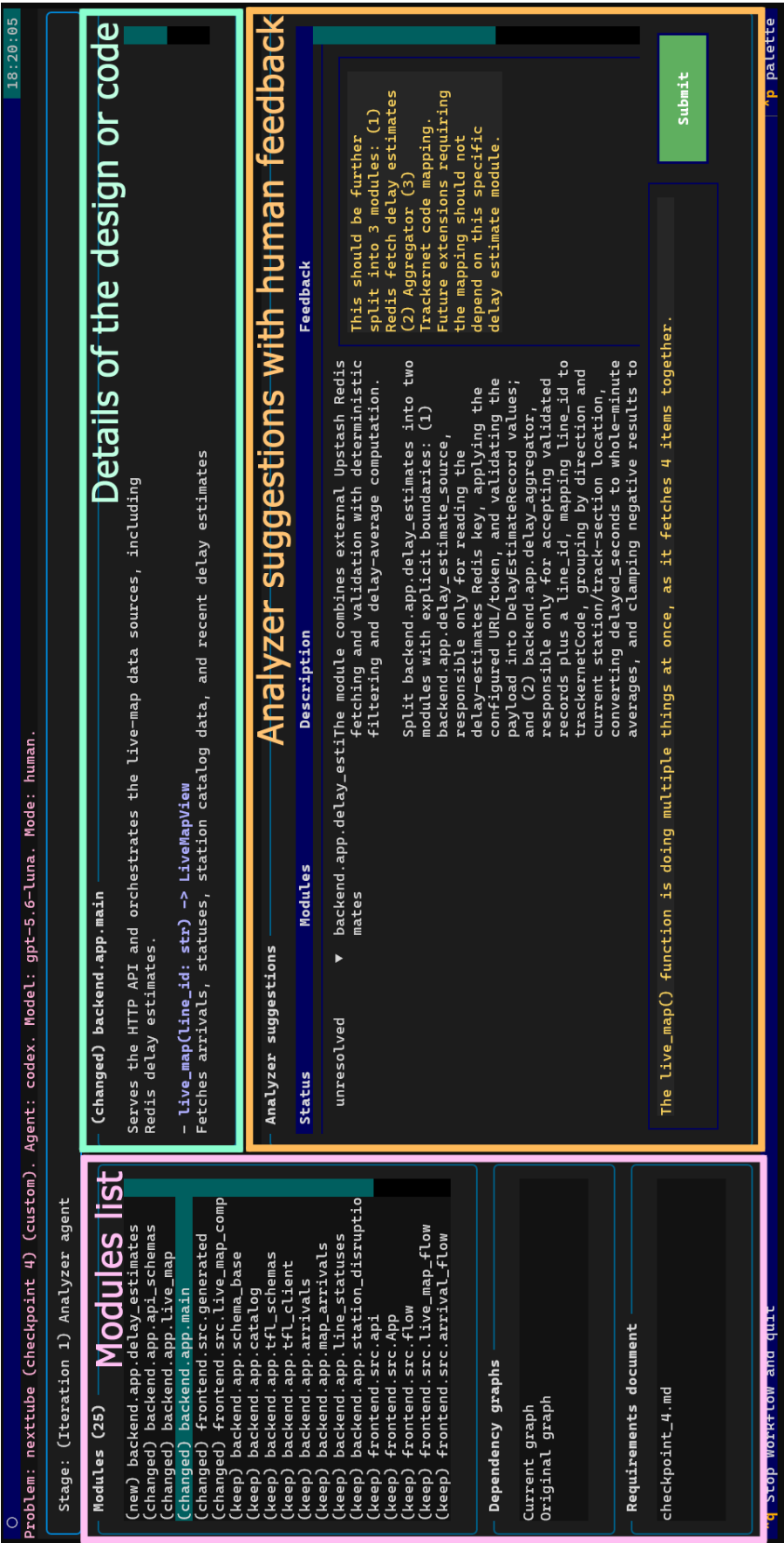


Figure 3.2: Textual UI user interface of the system.

the agents through the reasoning process on complex tasks, such as decomposing a system into modules with clear boundaries, or deciding when a decision requires additional input from the human.

Following the structured output approach adopted by RefAgent [39], prompts are returned as structured JSON format to be easily integrated into pipelines. However, instead of specifying its structure using natural language, we prompt the agent to only return JSON that matches a predefined JSON schema, as it is an effective way to restrict LLMs to only return the desired JSON output [41].

Agent communication is implemented through external JSON files, instead of using prompt chaining approaches where the output of the previous prompt is directly provided to the input of the next prompt. As outputs such as the design skeleton and analyser suggestions could grow substantially in size in later iterations of the loop, repeatedly propagating them through chained prompts may unnecessarily increase prompt size and introduce extra difficulty in state management.

The agents execute within a Bubblewrap bind-mounted workspace, which provides an isolated sandbox environment to restrict agent's read access to specified resources. The agents can only read the previous implementation, requirements document and the shared files, ensuring that their design decisions are not affected by external resources used in evaluation such as the test suites.

3.2 Decomposer Agent

The decomposer agent represents the first step of the pipeline, corresponding to the design stage of the software development life cycle. It takes as input the previous implementation if present and the requirements document. Based on this, it identifies module boundaries and decomposes the required functionality or feature addition into focused and independent modules, while discovering opportunities for module reuse from the existing implementation. Each module in the design is represented by its responsibility, dependencies and the public interface.

The agent is responsible for writing two structured JSON files at different levels of abstraction. The *high-level design* specifies how modules and data classes interact and communicate with each other in the form of a dependency adjacency list, by returning a list of modules indicating them as new, changed, kept or deleted. The *low-level design skeleton* describes the design specifics of each module in greater detail, including their responsibilities, non-responsibilities, expected input and outputs, preconditions and postconditions, and public interfaces including their parameters. This phase in the pipeline mainly focus on making architectural decisions rather than specifying implementation details, such as the private methods within the module. Specifically, the decomposer helps discover existing modules which encourage reuse, and identifies architectural smells such as centralised hub modules prior to implementation.

To achieve a balance between a monolithic and an over-modularised system, the

decomposer follows a set of guidelines when proposing modules following the discussion in the literature review. First, the number of modules proposed should be minimum yet preserving the single responsibility principle, avoiding intermediate wrapper modules that despite being individually cohesive, may increase the complexity in terms of readability and makes the code harder to understand. Second, modular decomposition should follow the principle of information hiding [29], with each module encapsulating as much complex logic as possible and exposing only the minimum of public methods. Third, the decomposer avoid making any assumptions about implementation details that have not been written. For example, it should not mark a module as insufficiently modularized because its anticipated implementation would be complex. Figure 3.3 shows the structure of the decomposer prompt, with the full prompt available in Appendix B.

```

You are a senior software engineer that decomposes requirements into modular design schemas.

You are working on the following issue: <previous_implementation> <requirements>
Prioritise reusing modules from the previous existing design (if any) and the previous implementation.
Incorporate analyzer suggestions in <current_analyzer_result.json> if any.

Propose a modular design that achieves the goal in the issue, following these principles:
<principles>

<Reasoning example on modular decomposition>

Task:
1. Create <current_design.json> representing the design skeleton, follow <design_json_schema>
2. Create <current_deps_graph.json> representing the dependency graph between modules, follow
<dependency_graph_json_schema>. Ensure the same module names are used across the design
and the dependency graph.

```

Figure 3.3: Structure of the decomposer prompt.

After the design and dependency graphs have been proposed, a deterministic validation step is present to ensure consistency and prevent hallucinated outputs, by ensuring that modules present in the design must also be represented in the dependency graph. Next, the workflow parses the agent-generated JSON files to identify a list of smells detectable solely from the design and dependency graphs alone, which are later read by the analyser. These include:

- Circular imports: Identified by cycle detection on the dependency graph. A circular import may indicate transitive dependencies between modules, suggesting the possibility of tight coupling. In addition, this may lead to partial initialisation of modules, making imports more prone to errors.
- Hub-like modularisation: A “hub” module has a high number of both incoming and outgoing dependency edges, known as high “fan-in” and “fan-out”. This increases maintenance burden as any change in the modules that it depends on would propagate to a high number of modules that depends on it.
- Unstable dependency: Instability is calculated by $I = \frac{C_e}{C_a + C_e}$ where C_a and

C_e denotes the afferent (Fan-in) and efferent coupling (Fan-out) respectively. The stable dependencies principle [42] states that modules should depend only on modules that are more stable than themselves, where Violations might introduce unnecessary coupling. For example, a module that depends on another less stable feature module only to import a shared data class, can potentially couple unrelated features and violate the interface segregation principle.

- Too many public methods: A large public interface may indicate insufficient encapsulation or exposure of unnecessary implementation details.

Detecting these structural design issues early in the pipeline, can reduce the need for substantial refactoring after the implementation.

3.3 Analyser Agent

The objective of the analyser agent is to critically evaluate the design proposed by the decomposer, or the implementation produced by coder agents. Although static analysis tools like DesignitePy or Pylint already produce a comprehensive overview of the code quality, relying on these tools are often insufficient to catch deeper structural issues such as duplicated logic, multiple responsibilities in the same module, and unnecessary dependency edges, as they require natural language understanding. Moreover, as discussed previously, static analysis tools are prone to false positives and provide a limited overview on the architectural quality.

Therefore, the LLM is explicitly instructed to first read the design or code and identify the structural issues following a set of criteria, and only use the identified smells to support its claims. The set of criteria used ensures that each smell is backed up by a real maintainability issue. First, modules identified as long or complex should be split if it is caused by a combination of distinct responsibilities, but not necessarily if the problem's nature justifies it, such as a parser entrypoint file that handles a large number of tokens without incorporating unrelated business logic. Second, the maintainability problems should be evidenced in the current design or implementation, rather than making assumptions about hypothetical future changes. This ensures that the agent is capturing real maintainability issues and is not biased by the raw set of identified metrics.

The human is involved in two stages within the analyser agent workflow. First, the analyser may ask additional questions that cannot be reliably inferred from the requirements, in order to justify a refactoring. The human would then review the questions and proposed trade-offs, and decide if the proposed refactoring is necessary. Second, after the analyzer agent generates a complete list of suggestions, the human can review them, reject inappropriate suggestions, and propose additional recommendations, as further described in section 3.4. Once all proposed suggestions have been resolved, the loop ends and the coder agent proceeds to implement the design or the code refactoring. Figure 3.4 shows the structure of the analyser prompt.

You are a senior software code quality analyst that gives refactoring suggestions.

You are working on the following issue: <requirements> <design or implementation>
<dependency_graph_json> <code_metrics_json>

Instructions:

1. Observe <design or implementation> and identify any antipatterns or maintainability issues discovered following <criteria>.
2. Read <code_metrics_json> and use them to support any code smells you have found.
3. For any potential issue, if there are extra contextual information that cannot be determined from the requirements alone, such as {...}, STOP and ask a human question in the following format:
<smell> <suggestion> <trade-offs>

<Reasoning example at the design stage to ask human a question>
<Reasoning example at the implementation stage to ask human a question>

4. Based on the smells identified and the contexts given, write improvement suggestions to <current_analyzer_result.json> structured in the json schema given by <analyzer_result_json_schema>. Simulate implementing all of them at once to ensure no new design issues are created.
5. Prompt the human for approval.

Figure 3.4: Structure of the analyser prompt.

3.4 Human Responses

As mentioned in Section 2.3.2, code quality metrics alone often fail to capture the full maintainability of software as they lack contextual information. One of them is volatility, which affects the maintenance effort (pain) defined by the product of strength, distance and volatility [28]. However, the expected frequency that a feature will change is not always available and accurate even when git history is available.

One factor that affects the volatility of a module is the subdomain that it belongs to: core subdomain represents features that give a competitive advantage and change most frequently, supporting subdomain are features that help the software function but does not give a competitive advantage, and generic subdomain are solved problems [28]. The subdomain of the affected modules can drastically affect the importance to resolve relevant code smells. For example, structural duplication of database CRUD operations may be acceptable since the operations are solved problems, but if the same operations process sensitive user information that must follow legal regulations that change over time, duplication may increase maintenance risk by requiring coordinated changes across multiple locations. Moreover, if these operations are expected to diverge in the future, resolving the duplication early can introduce “the wrong abstraction” [43], resulting in extra maintenance burden in the future.

Apart from the problem context, resolving code quality issues often involves trade-offs with no definite optimal solution, where the refactoring decision largely depends on the software team’s coding preferences. For example, breaking down a complex

function into private helpers may reduce the cyclomatic complexity, but increase the number of methods which may scatter the logic and make the code hard to trace. For example, if the team decides that the complexity is caused by null guarding rather than actual logic, it may be reasonable to keep the existing code as it remains sufficiently eadable.

In these cases, the analyser agent is encouraged to ask a human question rather than making implicit assumptions. We use two few shot examples to guide the analyser agent in recognising when human input may be required. The first example involves the decision to split a small text and date formatting module from data fetching, that depends on how frequently the functionalities are expected to change together, and involves a trade-off between separation of concerns and the additional complexity of introducing a small supporting helper module. The second example involves resolving duplication of an URL processing function, which depends on whether the implementations are expected to diverge in the future. Figure 3.5 shows the UI when the agent asks a human question.

In RefAgent [39], the feedback loop that terminates after the compiler agent succeeds and the tester agent passes all tests, or when 20 iterations are reached. However, our loop involves measuring the software quality which is a broad and subjective decision. Unlike compiling and testing whose outputs are objective and measurable, it is difficult to define a general threshold to decide on whether an analyser review should pass and coding should begin, or fail and require re-design. Although attempts have been made to establish quantitative quality thresholds, the observed results are inconsistent and often lead to over-modularisation.

As a result, the human is also involved in reviewing and approving the analyser suggestions at the end of each iteration of the feedback loop. The human can mark each suggestions as unresolved, accepted or rejected, provide feedback for each suggestion, and provide additional feedback that should be taken into consideration by the analyser. These reviewed suggestions are then incorporated into the shared analyser result document, allowing the analyser to refine its assessment based on the provided context and preferences. Once all suggestions have been resolved, the loop terminates.

3.5 Coder Agent

Once the coding stage begins, a list of files to be modified or created is identified from the design, which is then passed to the coder agent for implementation following the design structure. As the formatting of LLM-generated code may be inconsistent, the Prettier and Black formatters are applied to the code to ensure consistency and improve readability. After implementation, it then proceeds to re-generate the dependency graph and implementation-level metrics, which are read by the analyser in the coder-analyser critic loop.

In addition to the previous design-level metrics, tools including DesignitePy, Jscpd and Radon are used to generate implementation-level code quality metrics that can

17:25:55

Problem: nexttube (checkpoint 4) (custom). Agent: codex. Model: gpt-5.6-luna. Mode: human.

Stage: (Iteration 1) Analyzer agent

Modules (31)

(new) backend.app.delay_estimates

(changed) backend.app.api_schemas

(changed) backend.app.live_map

(changed) backend.app.main

(changed) frontend.src.generated

(changed) frontend.src.live_map_comp

(keep) backend

(keep) backend.app

(keep) backend.app.arrivals

(keep) backend.app.catalog

(keep) backend.app.line_statuses

(keep) backend.app.map_arrivals

(keep) backend.app.schema_base

(keep) backend.app.station_disruptio

(keep) backend.app.tfl_client

(keep) backend.app.tfl_schemas

(keep) backend

(keep) frontend

(keep) frontend.src

(keep) frontend.src.api

Dependency graphs

Current graph

Original graph

Requirements document

checkpoint_4.md

(new) backend.app.delay_estimates

Fetches the recent delay-estimate payload from Upstash Redis, maps a line id to its Trackernet code, and computes non-negative average minute delays for stations and route segments.

– fetch_delay_estimates(line_id: str, stations: list[CatalogStation]) -> DelayEstimates

Fetches and aggregates the delay-estimates Redis key for a line id and station catalog.

Human question

Smell: 'backend.app.delay_estimates' combines Redis fetching, line-code mapping, and delay aggregation.

Suggestion: Consider separating the Redis client, line-code mapping, and delay aggregation into focused modules while retaining 'fetch_delay_estimates' as the orchestration boundary.

Trade-offs: Splitting would isolate external I/O, configuration, and domain computation, but would increase module count and indirection for a single delay-estimate feature.

Do these responsibilities change independently enough to justify splitting 'backend.app.delay_estimates'?

Yes, they should be split, particularly line-code mapping that should belong to the data layer. Future extensions that depend on the line code mapping do not have to depend on delay estimates feature.

Submit

q Stop workflow and quit

^p palette

Figure 3.5: User interface when the agent ask human a question.

only be measured when the code is available. To prevent over-engineering and minimize bias towards the tools, only a list of selected subset of metrics relevant to broader maintainability concerns are exposed to the agent, instead of allowing it to read the raw files. These metrics include:

- **Cyclomatic complexity (CC):** The number of linearly independent execution paths. Values greater than or equal to 11 are reported as “worth inspecting”, and values greater than or equal to 21 are reported as complex functions, following Tom McCabe’s thresholds [44].
- **Lack of cohesion of methods (LCOM):** LCOM measures cohesion of a module depending on the sharing of instance variables between its methods, with greater sharing considered more cohesive. A commonly used implementation (LCOM2) defines P as the number of pairs of methods without accessing any common instance variables, and Q as the number of pairs that access at least one, with LCOM defined as $P - Q$ [45]. In this case however, Designite’s custom LCOM implementation between 0 and 1 is used with a threshold of 0.7, which better addresses the case when LCOM is not applicable [46]. Since it is prone to false positives when the number of methods is small, especially in utility classes, this metric is only reported when the module has ≥ 5 methods.
- **Feature envy:** A code smell where a method in one class is more interested in the data or methods of another class, which can indicate high coupling.
- **Protected access:** A code smell where the private fields or methods of another class is accessed. This can cause intrusive coupling, and risks silently altering another class’s behaviour through changes.
- **Insufficient modularisation:** A code smell reported by DesignitePy calculated using a combination of weighted methods per class (sum of CC of all methods), the number of methods and the number of public methods.

In addition, a summary comparing against the previous implementation in terms of lines of code, number of methods, cyclomatic complexity per function, and the duplicated proportion, is generated after implementation and can be read by later agents. The locations of duplicated lines reported by Jscpd is also given in the metrics json file, allowing the analyser agent to determine if the duplication represents a maintainability concern.

Chapter 4

Evaluation

4.1 Experimental Setup

In this section, we introduce the evaluation strategy for our multi-agent system, comparing its performance against direct LLM-based coding and against popular human-based GitHub repositories. Given the time and cost associated with evaluating the differences made by involving human-in-the-loop, the evaluation is separated into two parts: offline evaluation where we evaluate automatically against an iterative software development benchmark, and an in-depth analysis on the effectiveness of human-in-the-loop against expert-designed references on two specifically designed custom problems. We aim to answer the following research questions:

RQ1: How effective is our approach in mitigating the accumulation of technical debt in an iterative software development process?

RQ2: How does our multi-agent framework affect the functional correctness of LLM-generated code when compared to direct coding?

RQ3: How does the involvement of human-in-the-loop affect the design decisions made by LLMs?

4.1.1 Offline Evaluation

Benchmark Selection

To conduct an automatic evaluation, we would need to choose an LLM benchmark that can simultaneously measure the correctness of a major feature implementation without exposing the underlying test suite to the agent, as this would provide hints of the expected design structure and undermine the validity of measuring agent code quality. Table 4.1 lists benchmarks commonly used in LLM software evaluation but were not considered in our study.

Based on this, the SlopCodeBench [7] dataset was chosen for the offline evaluation, which features 36 problems spanning 196 checkpoints. For each checkpoint,

Table 4.1: Benchmarks considered for evaluation

Benchmark(s)	Reason They Were Not Chosen
HumanEval [47], MBPP [48]	They focus on LeetCode-style programming problems instead of major software implementations.
SWE-bench [14]	The benchmark is mostly associated with bug fixes instead of major feature additions.
FeatureBench [49]	While it focuses on end-to-end software development, it requires the code to match exactly a given interface in order to run Pytest correctly, leaking the design structure to agents.

the agent would need to implement the first implementation from scratch, then repeatedly modify and extend on its previous code. This implies earlier poor design decisions could accumulate technical debt over subsequent feature additions. The dataset features only command line interface (CLI) problems that are tested at its API boundary, meaning only the observable behaviours are tested (i.e. black-box testing). This allows the problems to be implemented using any programming language and any software design patterns. We selected 8 problems for our evaluation (*code search*, *circuit eval*, *etl pipeline*, *meshctl*, *mocked http*, *recli*, *sheeteval*, *sith*) spanning a range of difficulties and topics. Each problem consists of several distinct concerns that require appropriate separation, making them well suited for evaluating modular design capabilities.

To establish suitable baselines for the evaluation metrics, 8 public GitHub repositories written in Python ranging from 8.9k to 102k stars, with lines of code ranging from around 600 to over 5000, were chosen to analyze their source code directories (`src`, or equivalent). By inspecting the git history, historical versions corresponding to major releases were extracted, simulating the iterative development setting of SlopCodeBench for a fair comparison. The full list of selected GitHub repositories is listed in Appendix D.

Automatic Evaluation

To conduct an automatic evaluation without human involvement, the analyser prompt is modified to explicitly avoid making assumptions about ambiguous decisions that would otherwise require judgment from a human, and should leave the code as-is in these cases. Furthermore, to ensure that the decomposer–analyser and coder–analyser loops converge and eventually terminate, the maximum number of iterations of both loops are limited to 2. This limit is based on the assumption that each refactoring iteration is progressively improving code quality, without introducing major hidden design or coupling issues. Once the iteration threshold is reached, the loop exits and the system proceeds to the next stage. This revised system is illustrated in Figure 4.1.

Python is selected as the implementation language of SlopCodeBench problems, because of a wide range of tools supporting Python code quality evaluation and de-

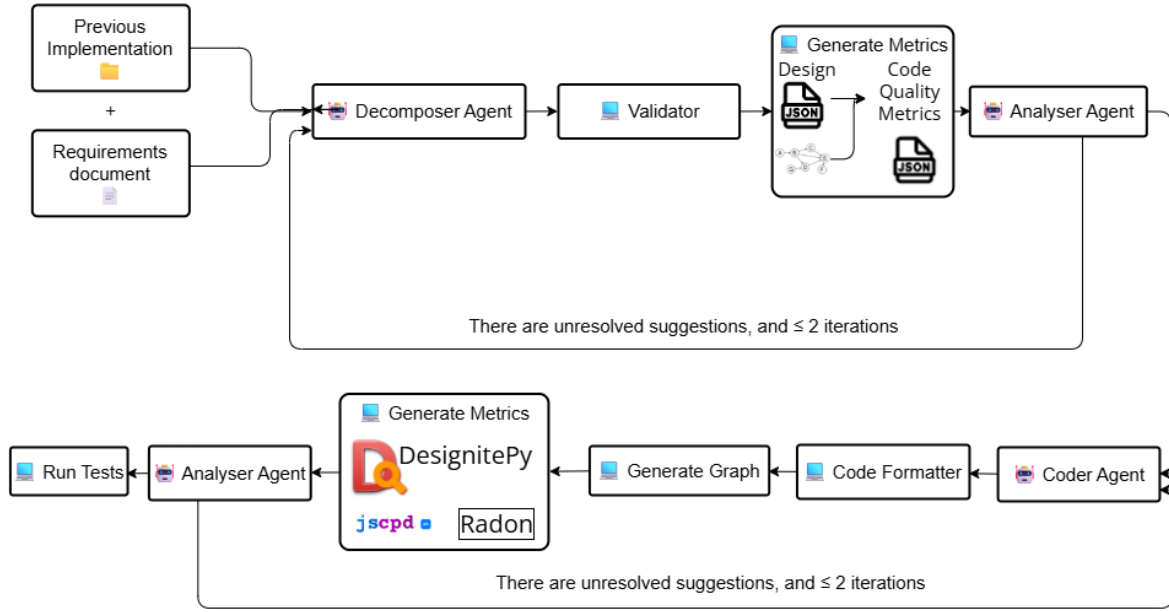


Figure 4.1: Multi-agent workflow modified for automatic offline evaluation.

pendency graph generation, and the author’s familiarity with the language. The GPT family of models are used in the LLM agents, particularly the latest state of the art GPT-5.6-Luna model featuring a 1M context window, and the GPT-5.4-Mini model, using the default reasoning level of medium. They were chosen as both are cost-effective models with exceptional coding ability, demonstrated by a 62.7% and 52.4% pass rates on the SWE-bench-PRO dataset, respectively [50, 51]. In addition, both models have knowledge cut-offs earlier than the release date of SlopCodeBench on March 25, 2026, reducing the risk that the benchmark was contaminated as part of the models’ training corpora.

Evaluation Metrics

A separate set of metrics are used to evaluate the code quality of the benchmark implementations, spanning aspects including readability, simplicity, maintainability, modularity and reusability. These includes:

- **Readability:** Logical lines of code (LLOC) per function and module, counting only executable statements. Overly long functions or modules may make them difficult to read and test.
- **Simplicity:** Mainly measured by the number of functions with cyclomatic complexity ≥ 11 . This count is used instead of the average, as the average is sensitive to the number of functions.
- **Maintainability:** The maintainability index (MI), measured by

$$MI = 171 - 5.2 \ln(V) - 0.23 G - 16.2 \ln(L)$$

[36] is used as it can be reliably computed, where halstead volume (V) measures the information density of a program based on its operator and operands,

G is the cyclomatic complexity and L is the lines of code. A weighted mean of MI over the lines of code for each file is obtained to form a single number.

- **Modularity:** The size of modules' public interfaces can detect if a module is exposing too much internals to others, which might indicate poor modularity and tight coupling. The average dependency graph degree and the propagation cost (PC) is used to indicate how coupled the system is, the latter metric measures the extent to which a change in a single element would potentially affect other elements [52]. To obtain this metric, a visibility matrix, V , is constructed from the transitive closure of the dependency graph, with $V_{ij} = 1$ indicates change in module i affects module j , either directly or indirectly, and $V_{ij} = 0$ otherwise. Propagation cost is then defined as the density of the matrix with n modules:

$$PC = \frac{1}{n^2} \sum_{i=1}^n \sum_{j=1}^n V_{ij}.$$

It is however, sensitive to the number of modules n , which determines the number of entries in the matrix [53]. Because of this, we also define impact size as the average number of modules affected by a change to a single module, which is the average number of non-zero entries per row in V . Mathematically,

$$\text{Impact Size} = PC \times n$$

- **Reusability:** Jscpd's reported duplicated lines and duplicated tokens proportion is used to indicate how reusable the code is, as extracting shared helper functions make the code more reusable.

Improvements in code quality may come with a cost of reduced functional correctness [7]. We therefore aim to evaluate the correctness of SlopCodeBench implementations under the multi-agent approach, measured by the benchmark's Pytest test suites. As each checkpoint should introduce new functionality while preserving behaviour from previous checkpoints, pass rates are evaluated separately against tests for the current specification, previous specifications, and the combined test suite.

4.1.2 Human-in-the-loop Evaluation

Traditionally, evaluating the impact of human-in-the-loop (HITL) in a multi agent system would involve surveying human participants with different levels of software engineering experience. Its effectiveness could then be assessed using metrics such as human design similarity, by comparing how well the architecture of the generated code aligns with human-designed software, or the human adoption rate by surveying their willingness to approve and adopt the generated code for production use. Although these metrics provide evidence on whether or not HITL improves the quality and acceptance rate of the software, recruiting a sufficiently large and diverse group of human participants is expensive in practice.

Therefore, this study evaluates HITL through two specialized and realistic software engineering problems, organised into incremental checkpoints following the format

of the SlopCodeBench dataset. In later sections, we would first introduce the problem checkpoints and the design principles, followed by an expert-designed modular decomposition, and simulate how a human would participate within the multi-agent system in practice. This study aim to investigate the effect on the overall software design with human involvement, by comparing it with the solution proposed by the expert and the offline approach. Although this approach may not provide a generalised measure of human acceptance rate, it demonstrates the potential benefits of human-LLM collaboration.

4.2 Evaluation Results

In this section, we present our results according to the research questions.

4.2.1 RQ1: How Effective Is Our Approach in Mitigating the Accumulation of Technical Debt in an Iterative Software Development Process?

Comparison with Coding Directly

To answer this research question, we first compare the evaluation metrics defined in Chapter 4.1.1, between directly generated LLM-based code and code generated using our multi-agent framework, on the 8 chosen SlopCodeBench problems. The results are compared across the GPT-5.6-Luna and GPT-5.4-Mini models. First, for each chosen problem, we extracted the relevant metrics per checkpoint. These metrics are then linearly interpolated to 8 checkpoints, which are then averaged per problem to visualize their average trends as the software evolves. In each table below, “start” and “end” denote the metric values at the first and last checkpoints respectively.

Readability. Table 4.2 shows the logical lines of code (LLOC), LLOC per function, LLOC per module, and the maximum lines of code for each function, across the two coding modes and two models.

Table 4.2: Comparison of readability metrics between direct coding and multi-agent approaches.

Mode	LLOC		LLOC per function		LLOC per module		Max LOC per function	
	Start	End	Start	End	Start	End	Start	End
Model: GPT-5.6 Luna								
Direct Coding	230	1168	21	24	230	1168	94	333
Multi-Agent	329	1724	13.5	12.1	60	50	47	87
Model: GPT-5.4 Mini								
Direct Coding	312	1538	14	18	312	1538	92	297
Multi-Agent	382	1108	13	10	86	94	59	94

The two models exhibit different behaviours when executing the multi-agent framework. For GPT-5.6-Luna, the logical lines of code of multi-agent is consistently higher than coding directly across every checkpoint and across all problems, with an average of 47.6% more lines written at the last checkpoint. That behaviour is less consistent for GPT-5.4-Mini, with the average logical lines of code of multi-agent initially higher, but becoming lower than direct coding from checkpoint 5 onwards.

For GPT-5.6-Luna, the higher logical lines of code can partly be attributed to the additional interfaces and wrappers created as a result of aggressive code modularisation. For example, in the *code_search* problem, the pattern matching behaviour is delegated from `pattern_matcher.py` to `pattern_engine.py`, where the former module is implemented as a thin adapter. In contrast, the main algorithm is directly implemented in `pattern_matcher.py` in the GPT-5.4-Mini implementation. Also, GPT-5.4-Mini seems to benefit more from the multi-agent approach through reduced duplication. For example in the same problem, the `parse_args()` and `parse_rules()` functions are duplicated in the direct coding implementation when their behaviour is extended by requirements of later checkpoint rules. This is not the case in the multi agent implementation as it reuses existing functionality.

The multi-agent approach consistently produces smaller functions across both models. The maximum function lines of code grows to $3.8\times$ and $3.16\times$ larger in the direct coding mode compared to the multi-agent mode, across the two models, respectively. Direct coding also consistently implement solutions concentrated in one long monolithic file, regardless of the problem difficulty and the number of checkpoints. As a result, its LLOC per module is much higher than that of multi-agent approach. Furthermore, both LLOC per function and LLOC per module are noticeably higher in the GPT-5.4-Mini model than GPT-5.6-Luna, suggesting that GPT-5.6-Luna tends to decompose the problem more aggressively into smaller and more focused modules when instructed to do so.

Simplicity. Table 4.3 shows the number of functions with high cyclomatic complexity ($CC \geq 11$) and the maximum CC of any function.

Table 4.3: Comparison of cyclomatic complexity between direct coding and multi-agent approaches.

Mode	Number of functions with $CC \geq 11$		Maximum CC of any function	
	Start	End	Start	End
Model: GPT-5.6 Luna				
Direct Coding	1.75	8	23.5	70.1
Multi-Agent	0.12	1.12	8.6	13.75
Model: GPT-5.4 Mini				
Direct Coding	1.88	10	18.88	58.63
Multi-Agent	0.38	1	9	10.13

The multi-agent approach consistently produces less complex functions by avoiding

the concentration of complexity, by isolating logic into private helper functions. Although the maximum function CC over the checkpoints increases by 59.9% from 8.6 to 13.75 in the GPT-5.6-Luna model, the maintenance risk is still relatively low, as no functions have CC greater than 20 according to Tom McCabe’s complexity thresholds [44]. In contrast, the maximum function complexity in the direct coding approach grows to 70.1 and 58.63 respectively, which is over $5\times$ that of our multi-agent approach. The results shows that our approach is effective at limiting the complexity of the program as development progresses, mitigating the iterative degradation that previously could not be adequately addressed through single-agent prompt modifications [7].

Maintainability. Table 4.4 shows the comparison of the weighted maintainability index across the two modes and two models.

Table 4.4: Comparison of weighted maintainability index between direct coding and multi-agent approaches.

Mode	Weighted Maintainability Index	
	Start	End
Model: GPT-5.6 Luna		
Direct Coding	22.21	0
Multi-Agent	42.98	42.69
Model: GPT-5.4 Mini		
Direct Coding	19.60	1.10
Multi-Agent	37.76	31.56

As the direct coding approach consistently concentrates all requirements and logic in the same file, the maintainability index drops quickly to 0 in the middle checkpoints. The multi-agent approach achieved a 94% and 93% higher initial weighted MI compared to direct coding, and they only dropped by 0.7% and 19.6% respectively towards the end. The weighted maintainability index is consistently above 20, indicating the multi-approach is more effective at preserving maintainability as additional functionality is introduced.

Modularity. Table 4.5 and 4.6 shows the modularity metrics across the two modes and two models. As the direct coding approach consistently produce single-module implementations, metrics such as a graph average degree of 0, and an impact size of 0 reflect their absence of modular decomposition rather than achieving the optimal values. In addition, none of the agent implementations produced code that contains dependency cycles, so that metric is omitted in the comparison.

The data further provided an evidence that GPT-5.6-Luna tends to decompose the design into modules more aggressively. Although they start with roughly the same initial number of modules, the GPT-5.6-Luna implementation has roughly $2.95\times$ more modules compared to GPT-5.4-Mini towards the last checkpoint. The size of the public interface is also lower than that of GPT-5.4-Mini, with each module only having 1.05 public methods on average towards the end.

Table 4.5: Comparison of dependency graph average degree and number of public methods per module between direct coding and multi-agent approaches.

Mode	Graph Average Degree		NOPM per Module	
	Start	End	Start	End
Model: GPT-5.6 Luna				
Direct Coding	0	0	3.71	10.56
Multi-Agent	1.82	2.75	0.73	1.05
Model: GPT-5.4 Mini				
Direct Coding	0	0	7.87	11.47
Multi-Agent	1.75	2.26	0.97	1.27

Table 4.6: Comparison of modularity and dependency metrics between direct coding and multi-agent approaches.

Mode	Number of Modules		Propagation Cost		Impact Size	
	Start	End	Start	End	Start	End
Model: GPT-5.6 Luna						
Direct Coding	1	1	1	1	0	0
Multi-Agent	5.9	35.1	0.47	0.20	2.12	5.37
Model: GPT-5.4 Mini						
Direct Coding	1	1	1	1	0	0
Multi-Agent	5.5	11.88	0.50	0.36	2.01	3.25

However, there is no direct evidence that the higher number of modules and a smaller average public interface size lead to extra coupling in the GPT-5.6-Luna implementation. Although the final impact size is around 65% higher than GPT-5.4-Mini, the propagation cost remains lower at 0.20 compared to 0.36. The average number of direct dependencies associated with each module, measured by the average degree of the dependency graph, is only 21.7% higher at the last checkpoint, despite GPT-5.6-Luna containing almost three times as many modules. This suggests that the additional decomposition does not result in a proportional increase in the inter-module coupling as the system grows.

Reusability. Table 4.7 shows the percentage of duplicated lines and tokens across the two modes and two models.

Table 4.7: Comparison of code duplication between direct coding and multi-agent approaches.

Mode	Duplicated Lines		Duplicated Tokens	
	Start	End	Start	End
Model: GPT-5.6 Luna				
Direct Coding	0	0.01	0	0.01
Multi-Agent	0	0	0	0
Model: GPT-5.4 Mini				
Direct Coding	0	0.041	0	0.044
Multi-Agent	0	0.007	0	0.009

Although the proportion of duplicated code remains low for both direct coding and multi agent, the direct coding approach is more likely to duplicate complete functions instead of reusing them as new features are added. For example, in GPT-5.6-Luna’s implementation of *code_search*, the *width_expr()* function is duplicated across two evaluate functions, each of which hard codes the same list of operators in a conditional, where changes in that list would require coordination.

While multi-agent reduced the proportion of duplicates in both models, the reduction is more significant in the GPT-5.4-Mini model, reducing final duplicated lines and tokens by 3.4% and 3.5% respectively. It can therefore be concluded that the multi-agent approach consistently produce less duplicated code, leading to more reusable functions that allows for better independent changes.

Comparison with Popular GitHub Repositories

We further compare these metrics against 8 popular GitHub repositories as references to investigate two questions: (1) Whether the directly generated agent code exhibits lower quality than human-written code, and (2) whether our proposed multi-agent approach produces code whose quality align more closely with human-written software. To ensure the results are not affected by the code formatting style, the Black formatter is applied to all Python source code in the selected repositories, in order to stay consistent with the formatting applied to agent-generated code.

We present our results of the 5 code quality aspects below:

Readability. The logical lines of code of the reference GitHub repositories ranges from 378 (*Uvicorn*) to 3172 (*Flask*) in the first checkpoint, and from 625 (*Python-dotenv*) to 5739 (*Click*) in the last checkpoint. Because no SlopCodeBench implementations exceeds 2000 LLOC, we mainly focus on the size of functions and modules for this evaluation. Figure 4.2 shows the comparison between GPT-5.6-Luna and GPT-5.4-Mini for direct coding and multi-agent and the reference repositories, which are indicated in dotted lines.

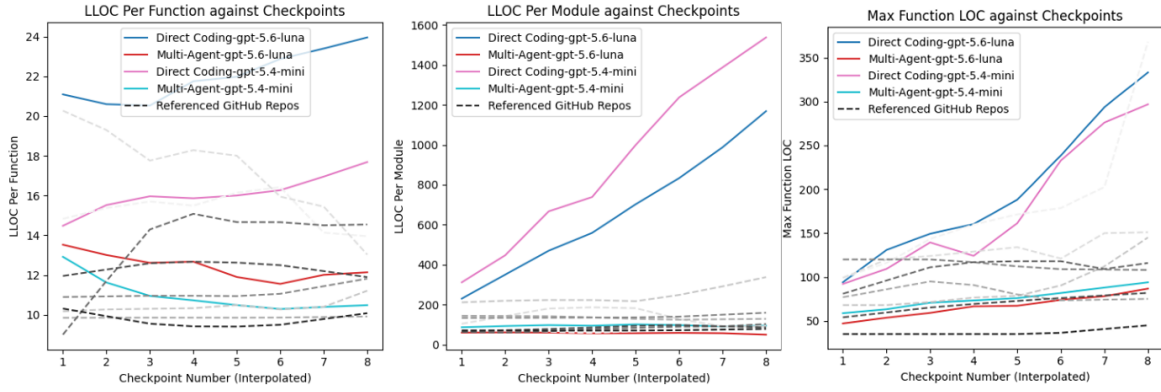


Figure 4.2: Comparison of LLOC per function and module, between agent code and reference repositories.

Directly generated agent code exhibits substantially higher LLOC per function and per module compared to reference repositories, with the maximum function lines of code being approximately $2.31\times$ higher than the repositories average, when averaged across the two GPT models. In the multi-agent approach however, the averaged max LOC of function is then 90.4, lower than 136.25 of the repositories, and the LLOC per function is also slightly lower at 11.31 compared to 12.06. The results demonstrate that the multi-agent approach produces function and module sizes that more closely align with those observed in human-written software.

Simplicity. Figure 4.3 shows the number of high cyclomatic complexity functions ($CC \geq 11$) and their function maximum, across agent code and reference repositories.

Although direct agent coding produces an average of 8 high complexity functions at the final checkpoint, which is slightly lower than the repository average of 9.5, its maximum function cyclomatic complexity increases sharply after checkpoint 5 and exceeds the value observed in all reference repositories. On the other hand, the multi-agent code complexity stayed well below the average of the repositories, indicating that agent-generated code indeed accumulates complexity quicker than human-written code, and our multi-agent approach is effective at mitigating this phenomenon.

Maintainability. Figure 4.4 shows the weighted maintainability index across agent code and reference repositories.

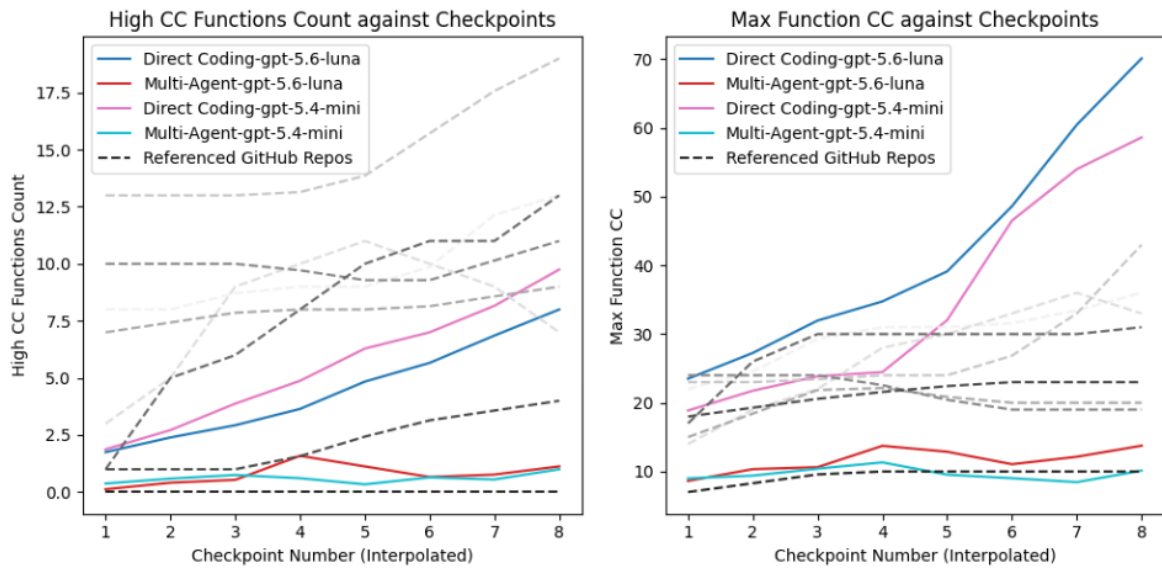


Figure 4.3: Comparison of cyclomatic complexity between agent-generated code and reference repositories.

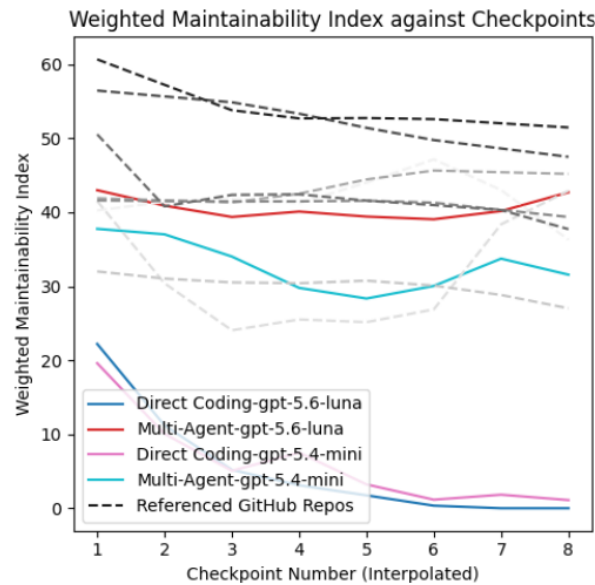


Figure 4.4: Comparison of weighted MI between agent-generated code and reference repositories.

The maintainability index drops for both agent and human-written code, by 8.0% and 10.2% respectively. Specifically, the average value decreases from 40.4 to 37.1, and from 45.6 to 40.9 respectively. As previously mentioned, since direct agent coding consistently use one monolithic module, both its initial and final maintainability index are much lower than that of GitHub repositories and the multi-agent mode. Therefore, the results show that our multi-agent approach produces code with the maintainability more closely align with those of human repositories.

Modularity. Figure 4.5 shows the public interface size, and Figure 4.6 shows the number of modules and propagation cost, across agent-generated code and human-written repositories.

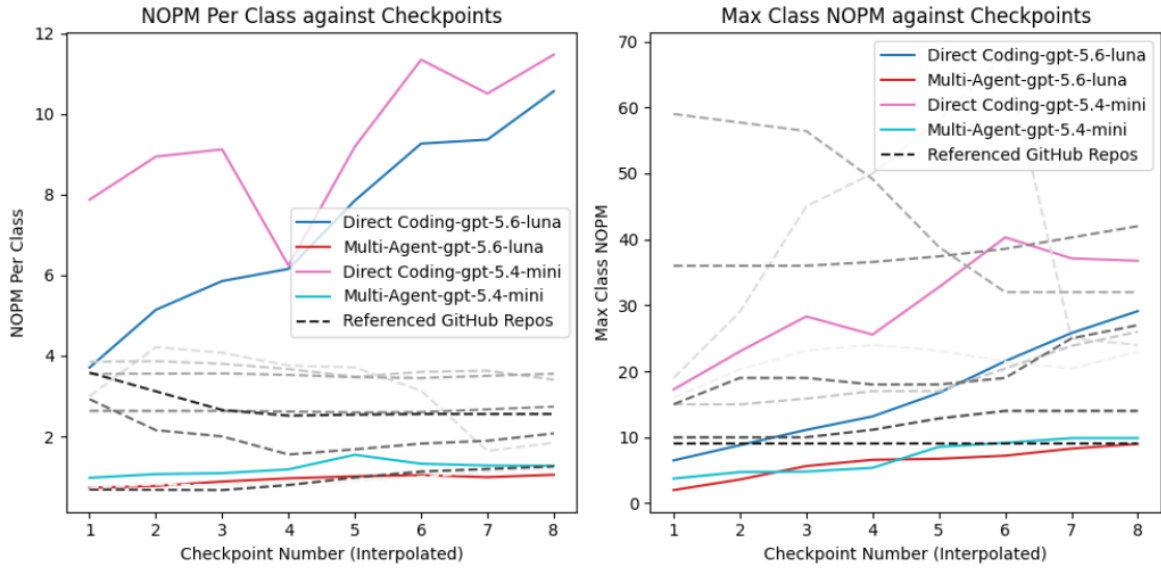


Figure 4.5: Comparison of the public interface size between agent-generated code and reference repositories.

Both sets of data suggest that our proposed multi-agent approach tends to encourage LLM agents to decompose code into smaller, more focused modules more aggressively than typically observed in human-written code. The average public interface size of the multi-agent approach is 1.16 at the last checkpoint, lower than the average value of 2.33 in human repositories. Furthermore, the average number of modules in the multi-agent mode increases by $4.13\times$ from 5.69 to 23.5, whereas that only increases by $1.94\times$ in human repositories. This rapid increase in the module count is likely what caused the slight drop in propagation cost, while that of human repositories remains comparatively stable across checkpoints.

Reusability. Figure 4.7 shows the proportion of duplication across agent and human-written code.

Although the figure shows that the final duplicated lines and duplicated tokens of human-written repositories are on average higher than all agent-generated code, they do not necessarily indicate duplication of business logic. For example, *fastapi* duplicates the public API across `application.py` and `routing.py`, however elimi-

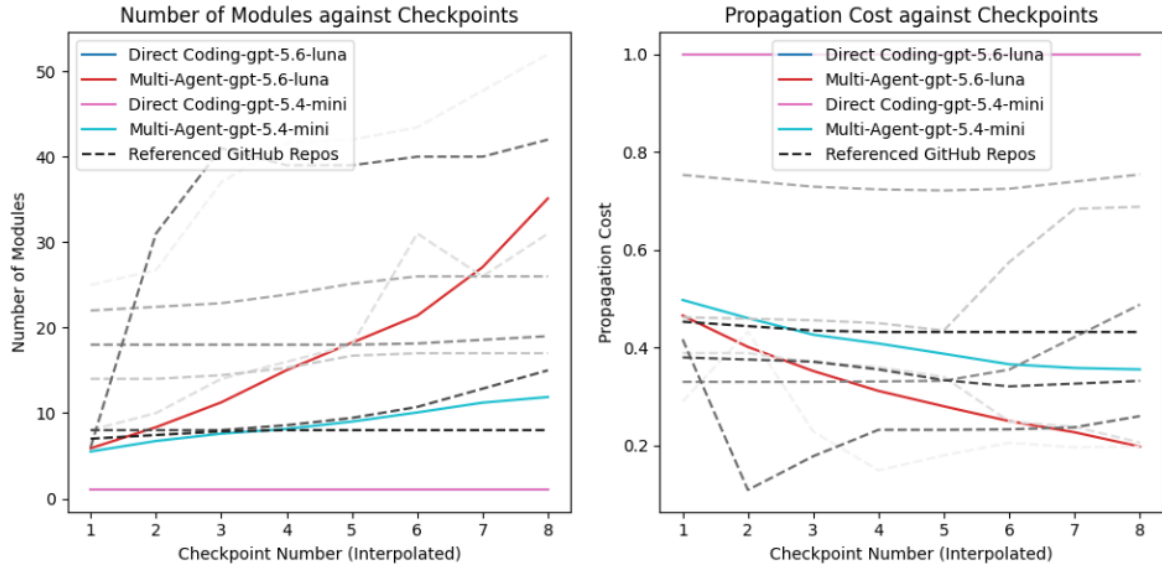


Figure 4.6: Comparison of the number of modules and the propagation cost between agent-generated code and reference repositories.

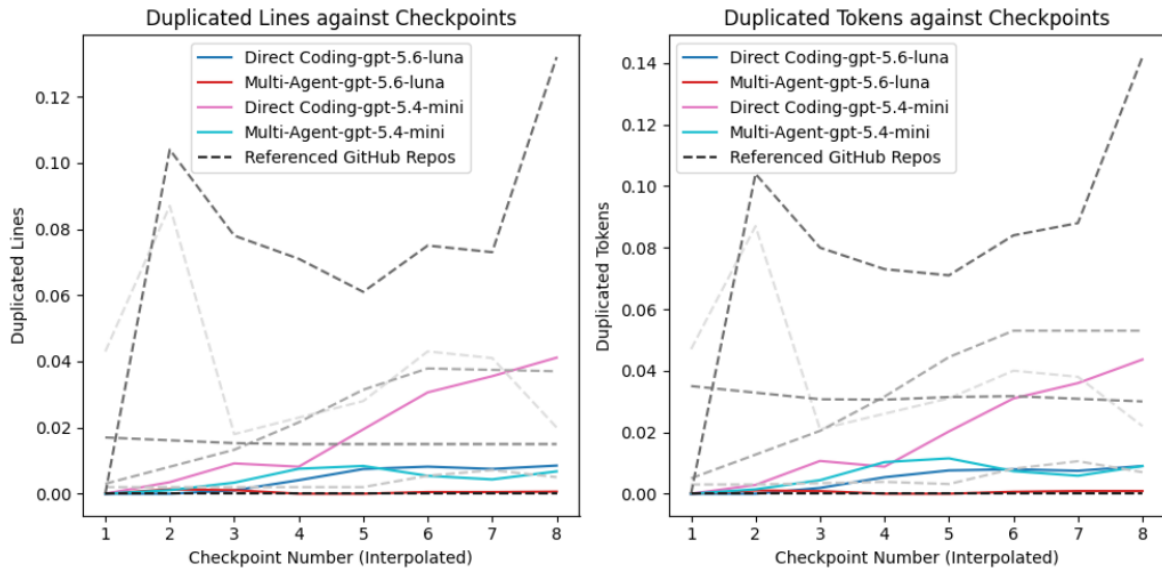


Figure 4.7: Comparison of the duplicated lines and tokens between agent-generated code and reference repositories.

nating this duplication by dynamically generating fields might not necessarily lead to better readability. Although the absolute duplication values may not be directly comparable, nevertheless, the rate of duplicated lines in GPT-5.4-Mini direct coding increased by 4.1% over the checkpoints, the human-written code average of 3%. In contrast, the multi-agent approach is more effective at limiting the accumulation of duplicated code as the software becomes more complex.

Answer to RQ1: The offline evaluation suggests that our multi-agent approach is effective at mitigating the accumulation of technical debt across the 5 software quality aspects. Compared with directly generated agent code, the framework prevents the concentration of function complexity, avoids functions that rapidly grow in size, and reduces duplicated code, by decomposing the system into modules that are focused and more loosely coupled. Ultimately, this leads to a more maintainable codebase, with the maintainability index aligning more closely with the values observed in human-written GitHub repositories.

However, the behaviour of our multi-agent approach is not entirely consistent across different LLM models. For example, GPT-5.6-Luna produced 47.6% more code on average compared to direct coding, but that behaviour is not observed in the GPT-5.4-Mini model. In addition, LLMs tend to decompose into smaller modules more aggressively than humans do when encouraged to prioritise modularity, indicated by the rapidly growing number of modules and their relatively small public interface size. These inconsistencies provided further evidence and motivation for human involvement in the framework, by going through an additional review stage so the design decisions can be assessed against the subjective project preferences.

4.2.2 RQ2: How Does Our Multi-Agent Framework Affect the Functional Correctness of LLM-Generated Code When Compared to Direct Coding?

As mentioned in chapter 4.1.1, improved code quality may come with the cost of reduced functional correctness. As a result, we augmented our multi-agent framework with a test-driven development (TDD) approach in an attempt to preserve functional correctness while benefiting from the improved code quality. Based on the red–green–refactor cycle [54] of TDD, two additional specialised agents are introduced:

- **Test planner agent:** This agent is responsible for designing a separate Pytest test suite before implementation, corresponding to the “red” stage of TDD. Given the requirements document and the existing code, the agent generates tests only for explicitly defined behaviours and edge cases, listed in the requirements for the current checkpoint. After tests generation, the agent would simulate running the tests to ensure they fail initially before implementation. To avoid leaking the test suite to the decomposer agent, the tests are temporar-

ily moved out of the bind-mounted agent workspace.

- **Tester agent:** After the implementation has been completed, before any refactoring, the test suite is restored to the agent workspace. The tester agent would then read the implementation code and the test suite generated by the test planner agent. It has the ability to execute the tests and perform minimal modification to the the code to resolve any failures, corresponding to the “green” stage of TDD. After the tests pass, the analyser agent would then refactor the code based on the implementation metrics, corresponding to the “refactor” stage. The tester is involved in every loop between the coder and analyser to ensure the tests still pass over the code quality refactoring attempts.

This revised architecture in our offline evaluation is illustrated in Figure 4.8. Table 4.8 presents the performance of each code generation mode across two models, when evaluated against SlopCodeBench’s test suites against the current checkpoint, previous checkpoints, and their combined results.

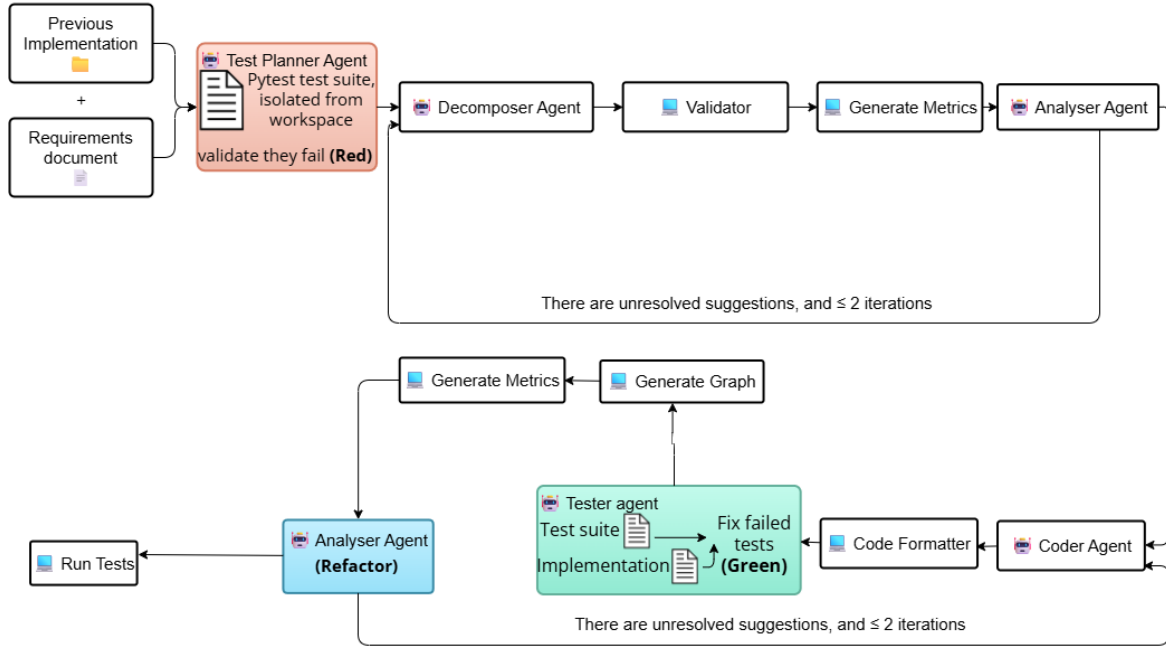


Figure 4.8: Multi-agent system workflow modified with TDD by introducing two new agents.

The results show that our multi-agent approach does indeed reduce the functional correctness relative to direct coding, across both GPT models. Although the GPT-5.6-Luna model consistently outperforms the GPT-5.4-Mini model in all modes, our multi-agent approach reduced the functional correctness of both models by 25% and 19% respectively when tested against the current checkpoint, and by 24% and 19% respectively in all tests.

Our revised test-driven development approach partially mitigates this trade-off by improving the overall pass rate by 9% for the GPT-5.6-Luna model, but it further reduced the code correctness by 19% for the GPT-5.4-Mini model. However, the

Table 4.8: Comparison of test pass rates between direct coding, multi-agent, and multi-agent with TDD approaches.

Mode	Current Checkpoint		Previous Checkpoints		All Checkpoints	
	GPT-5.6 Luna	GPT-5.4 Mini	GPT-5.6 Luna	GPT-5.4 Mini	GPT-5.6 Luna	GPT-5.4 Mini
Direct Coding	0.82	0.59	0.85	0.66	0.83	0.63
Multi-Agent	0.57	0.40	0.64	0.50	0.59	0.44
Multi-Agent+TDD	0.65	0.23	0.71	0.32	0.68	0.25

improvement on GPT-5.6-Luna is not entirely consistent. For example, TDD more than doubles the final combined test pass rate across all checkpoints of the *sheeteval* problem, from 0.323 to 0.677. However, in *recli*, the TDD approach achieved a test pass rate of 0.494, slightly lower than that of multi-agent at 0.505, both lower than the pass rate of direct coding at 0.659. The overall improvement brought by TDD, is however, not significantly observed across all problems in the GPT-5.4-Mini model.

Answer to RQ2: Our multi-agent approach exhibits a trade-off between the improved code quality and reduced functional correctness across both models. To address this trade-off, we revise the architecture to feature test-driven development (TDD), introducing two additional agents executing the red-green-refactor cycle. While this modification improves the test pass rates for the GPT-5.6-Luna model by around 9%, it further reduces the performance of GPT-5.4-Mini. This shows that the effectiveness of this revised approach is model-dependent, and encouraging the model to write tests before implementation does not necessarily lead to improved performance, particularly for less capable models.

4.2.3 RQ3: How Does the Involvement of Human-in-the-Loop Affect the Design Decisions Made by LLMs?

In the following section, we first present the requirements for each checkpoint of the problems designed for human-in-the-loop evaluation, followed by a discussion on how their characteristics facilitate the evaluation of modular and maintainable software design. Next, we propose an expert-designed reference architecture for the focused parts of each problem. We then examine whether the full multi-agent workflow effectively mitigates the maintainability issues that arise when the modules are not appropriately decomposed and reused, and analyse the quality of questions asked by the agent to determine how human-in-the-loop influences the final design decisions. Finally, these decisions are compared against the implementation using the offline approach, and against the reference expert designs, to investigate if human-in-the-loop can guide the resulting designs towards those preferred by a human expert. The GPT-5.6-Luna model is used because of its stronger ability to pro-

duce correct solutions. The full problem descriptions are available under the project repository given in Chapter 5.2.

Problem 1: Iris Recognition CLI Tool

Problem Introduction. This problem requires implementing a command line tool for automatic subject identification when given an iris image, through the iris recognition pipeline presented by Daugman’s method [55]. The implementation is based on the CASIA-Iris-Interval dataset [56], featuring high-quality grayscale near-infrared images of the eye on 249 subjects, which is provided in a separate data directory. Each checkpoint represents a major component of the pipeline:

- Checkpoint 1 involves **dataset discovery**, to scan and validate the CASIA dataset structure, inspect individual subjects, and report anomalies such as corrupted images to a csv file.
- Checkpoint 2 involves **iris localisation**, to identify the inner and outer circular boundaries using the integrodifferential operator [57]. It reads the previously generated csv and writes the segmented output and anomalies to external csv files.
- Checkpoint 3 involves **normalisation** by transforming the segmented iris region into a rubber sheet model with fixed dimensions to allow for comparison [58]. It reads the segmentation csv file and similarly writes anomalies.
- Checkpoint 4 involves **encoding** which encodes the normalised image by applying 2D gabor wavelets and converting to iris code [58]. It outputs the encoded and masked bin files to two folders, matching the structure of the dataset, and similarly reports anomalies.
- Checkpoint 5 and 6 are for **matching and subject recognition** by iris code matching. The closest target is determined with the shortest hamming distance smaller than a threshold. The `<identify>` command, accepts an image path and perform all previous steps to generate an iris code for comparison, in order to identify the closest subject.

Figure 4.9 presents an example of expert-designed modular decomposition written by the author. Rather than assigning a module per pipeline step, each stage is composed of the core algorithm module (integrodifferential operator, polar coordinates for normalisation etc.), the anomaly detectors, data models and a service coordinator. The entrypoint simply accepts arguments and calls the service modules. In addition, as the csv columns defined in the previous step are reused in later steps, they are extracted in a shared schema module to avoid duplication. Similarly, repeated dataset folder structure traversal and image validation are extracted as separate reusable modules.

For this system, a monolithic architecture could introduce substantial technical debt. For example, duplication of csv columns and folder traversal logic would require coordinated changes, if the underlying dataset is modified. Furthermore, as the fi-

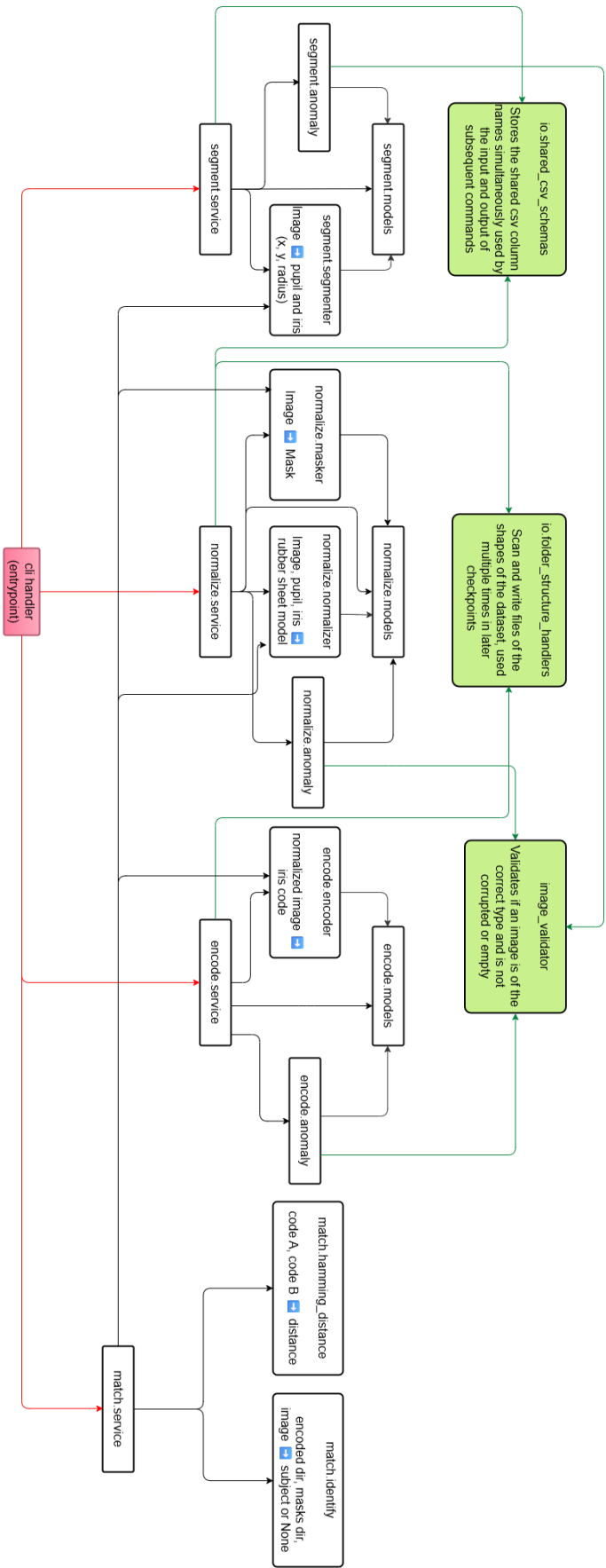


Figure 4.9: Expert-designed modular decomposition of the iris recognition problem.

nal matching stage depends only on the business logic of multiple preceding stages, failing to separate logic from IO risks coupling unrelated functionalities together. Therefore, this problem provides a suitable case study for human-in-the-loop evaluation on whether the above concerns are addressed.

Results. Table 4.9 presents a selection of questions raised by the analyser agent and the corresponding human responses. For conciseness, the agent’s questions have been condensed while preserving their meaning. A more comprehensive set of questions asked can be found in Appendix C.

Table 4.9: Examples of agent–human interaction across checkpoints.

Checkpoint	Examples of Agent–Human Interaction
Checkpoint 3 (Normalisation)	Agent segmentation_service and normalisation_service duplicate CSV schema field definitions. Extract shared fields into a constants module and import them. This would remove duplication, but may centralise schemas that may intentionally evolve independently. Human The CSV schemas should be synchronised rather than duplicated, as the CSV columns are consumed by later pipeline stages.
Checkpoint 4 (Encoding)	Agent CSV anomaly serialisation is duplicated across csv_writer, encoding_csv_writer, normalisation_csv_writer, and segmentation_csv_writer. Extract the shared anomaly CSV writing logic into a common helper. This would reduce repeated code, but changes to the shared helper would affect every stage of the pipeline. Human It is not worth extracting because the anomaly helper is small, and the anomaly fields for each pipeline stage may diverge in the future.

Continued on next page

Checkpoint	Examples of Agent–Human Interaction
	Agent <code>segmntnation.service</code> and <code>normalisation.service</code> each implements <code>_identity(path)</code> to derive the subject and eye identifiers from an image path. Extract the shared identity logic into a small utility module. This would remove duplicated behaviour but centralises a rule that becomes inconvenient if they later require different identity semantics. Human As the folder structures will remain identical across different stages, the duplicated path identity function should be extracted.

The results show that with the human-in-the-loop approach, it effectively guides agents to raise meaningful code quality questions, supported by the proposed refactoring and the trade-offs that arise. The analyser agent is repeatedly able to continuously seek relevant contextual information and clarify design goals with the human. Particularly, before recommending abstractions to eliminate duplication, it is effective in identifying the context and assessing whether duplicated functionality would diverge over time, or if the repeated code is insignificant to justify introducing a new helper module. To further access the differences compared to the automatic offline multi-agent code generation, Table 4.10 compares how these design decisions are implemented across the offline and the human-in-the-loop approach.

The results show that with human-in-the-loop, it is effective at reducing duplication and improving reusability, such as the extracting the csv column values shared between subsequent stages, aligning the solution more closely with the expert-designed architecture presented in Figure 4.9. As the agent is aware that extracting reusable functions may lead to “the wrong abstraction” [43], it consistently consults and inform the human regarding the trade-offs of these refactoring decisions. As mentioned in 4.1.1, agents in the offline approach are told not to make assumptions about ambiguous context information, thus duplication remains in their implementation. This may increase the risk of coordinated changes when the duplicated logic is updated, such as by introducing a new iris dataset with a different directory structure.

Problem 2: London Underground Arrivals App

This problem requires implementing a full-stack web app showing London Underground information extracted from the Transport for London public API, using Python and FastAPI for the backend and React TypeScript for the frontend. It consists of 4 checkpoints, each one adding features that may require extension on previous fea-

Table 4.10: Comparison between design decisions of the iris recognition problem across the offline and human-in-the-loop approach.

Feature	Offline Implementation	HITL Implementation
Segmentation and anomaly CSV columns serialisation	Each stage of the pipeline owns a separate module dedicated to csv reading and writing. Both output and anomaly CSV columns are defined as constants in each file, with the column names duplicated across consecutive stages.	Although each stage also owns a separate csv writer module, the dataset and segmentation csv columns are extracted as a separate module to avoid duplication. The anomaly csv columns are intentionally kept separated.
Identify the subject and eye identifiers (Left or Right) from image path	This functionality is implemented independently using different code, and is duplicated across <code>encoding_workflow.py</code> and <code>identification_workflow.py</code> .	This functionality is intentionally split into a shared helper in <code>path_identity.py</code> , allowing consistent modification when another dataset is used.
The final identify command that integrates all previous stages of the pipeline	The IO is separated from the business logic at each stage, with the identification workflow effectively reusing existing functions.	Similarly, the IO and business logic are naturally separated and are effectively reused at the identification stage.

tures:

- Checkpoint 1 implements **station search** by name and derived 3-letter code, and **train arrivals display** with platform and direction metadata and deduplicated entries by their vehicle IDs.
- Checkpoint 2 involves **status and disruptions display**, involving formatting line and station disruptions.
- Checkpoint 3 involves creating **line maps**, reusing status fetching to highlight sections, and reusing arrival fetching feature to determine vehicle locations through parsing a location string.
- Checkpoint 4 involves displaying **delay estimates** on the map, involving a new fetch mode from Redis that reuses location parsing in live maps but uses a new line ID mapping.

This problem is designed to place particular emphasis on reusability across successive feature additions. Checkpoint 3 introduces line maps, which reuses only parts of the arrival board functionality, particularly requiring the string indicating the current location of vehicles, and requiring part of the line disruption module to highlight affected sections. The same location parsing method, is reused later in checkpoint 4. These design principles are illustrated in Figure 4.10, showing an expert-designed architecture focusing on the components most relevant to the maintainability concerns discussed.

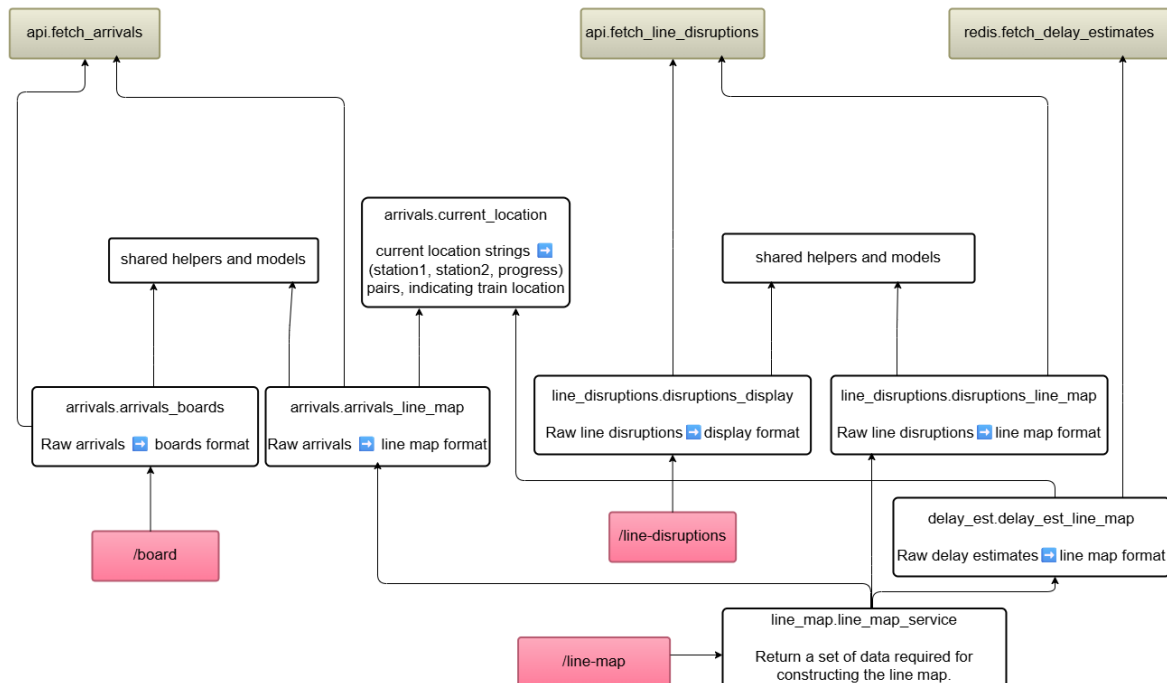


Figure 4.10: An example of expert-designed modular decomposition of the London Underground App problem.

Similarly, we evaluated the design decisions made by the offline multi-agent work-

flow and the human-in-the-loop workflow, with their differences presented in table 4.11.

Table 4.11: Comparison between design decisions of the Underground App problem across the offline and human-in-the-loop approach.

Feature	Offline Implementation	HITL Implementation
API data models interfaces implementation across the backend and frontend	Overlapping API models are defined independently across the Python backend and the TypeScript frontend.	As the agent raised concern about this duplication that would cause inconsistency, API data models are defined in <code>backend/api_schemas.py</code> as the single source of truth, with a script to generate the frontend equivalents.
Line disruption severity extraction in live maps	The offline workflow reused the <code>build_line_statuses</code> method of previous checkpoints in the <code>build_live_map</code> function, effectively reusing behaviour.	The severity extraction is implemented as a standalone private function, resulting in the high complexity. As the agent flagged the issue and recommended creating additional private helpers to reduce the complexity, the human initially rejected this suggestion. Further investigation revealed the underlying issue was a missed opportunity to reuse existing line disruptions behaviour.

Based on the reusability aspect we focus on, we further examined how the delay estimates component reuse functionality from previous checkpoints across both modes. We found that both the offline and HITL approach duplicated the non-trivial location string parsing logic across checkpoint 3 and 4, and as a result, the human provided feedback at the approval stage that decides if the decomposer–analyser or coder–analyser loop should continue:

Human

The “current location” string parsing that determines the location of the train vehicle, is duplicated between the live map to show train locations and the delay estimates module to show delay locations. You should extract that as a shared helper.

Agent

Added one new analyzer suggestion: The current location parsing logic is duplicated between `map_arrivals.py` and `delay_estimates.py`. Therefore, add a shared `location_parser` module that owns station resolution and current location parsing.

The human, who acts as the coordinator, is able to add or remove analyzer suggestions, which better aligns the code with human preferences. Based on the observations, we answer our third research question below.

Answer to RQ3: Human-in-the-loop enables agents to ask targetted rather than generic questions, ensuring more reliable outcomes when addressing design decisions that require additional context, human judgment, or quality trade-offs. For example, the agent might ask whether duplicated components would diverge in the future before determining whether abstraction is appropriate, minimising the risks of “the wrong abstraction”. The human is kept continuously informed about the refactoring decisions made by the agent, and act as a coordinator to determine if each change is justified, reducing the risk of over-engineering. Using two specialised, real-life software engineering examples and the expert-designed modular decomposition as analysis, we demonstrated the potential for the human-in-the-loop approach to guide agents towards design decisions that are more closely aligned with human preferences.

4.2.4 Ablation Study

In the above evaluation, some overlapping metrics are used as both the optimisation objective and the evaluation metrics. For example, “lines of code” and “cyclomatic complexity” are used in guiding the analyser agent to identify refactoring opportunities after the coder has finished its implementation, but the same set of metrics are part of the calculation of the maintainability index (MI) used to evaluate software maintainability. As a result, there is a risk of Goodhart’s law that states “When a measure becomes a target, it ceases to be a good measure”. [59]. To investigate this potential effect, we conducted an ablation study on the SlopCodeBench dataset by removing isolated parts of the system, to assess whether the quality improvements remain consistent when the metrics generated by DesignitePy, Jscpd and Radon are no longer provided after the implementation stage, as these metrics are primarily used in the evaluation. Specifically, we kept the metrics generated from the dependency graphs alone in the design stage, but removed metrics generation in the implementation stage, as indicated in Figure 4.11.

Figure 4.12 and 4.13 show the cyclomatic complexity metrics and the weighted maintainability index respectively, in the direct coding, multi-agent, and the multi-agent mode with implementation metrics removed modes. We specifically investigate the effect on these metrics as they are directly provided as part of the implementation metrics, available to the analyser agent.

The results show that there is a substantial improvement in code quality and maintainability, even without the implementation metrics provided, with the effect of removing them more evident in GPT-5.6-Luna than GPT-5.4-Mini. Although GPT-5.6-Luna generated an additional 5.6 functions with cyclomatic complexity ≥ 11 , that value remains below the 8 high complexity functions observed under direct coding. The final maximum function CC in GPT-5.6-Luna is around 82.7% higher

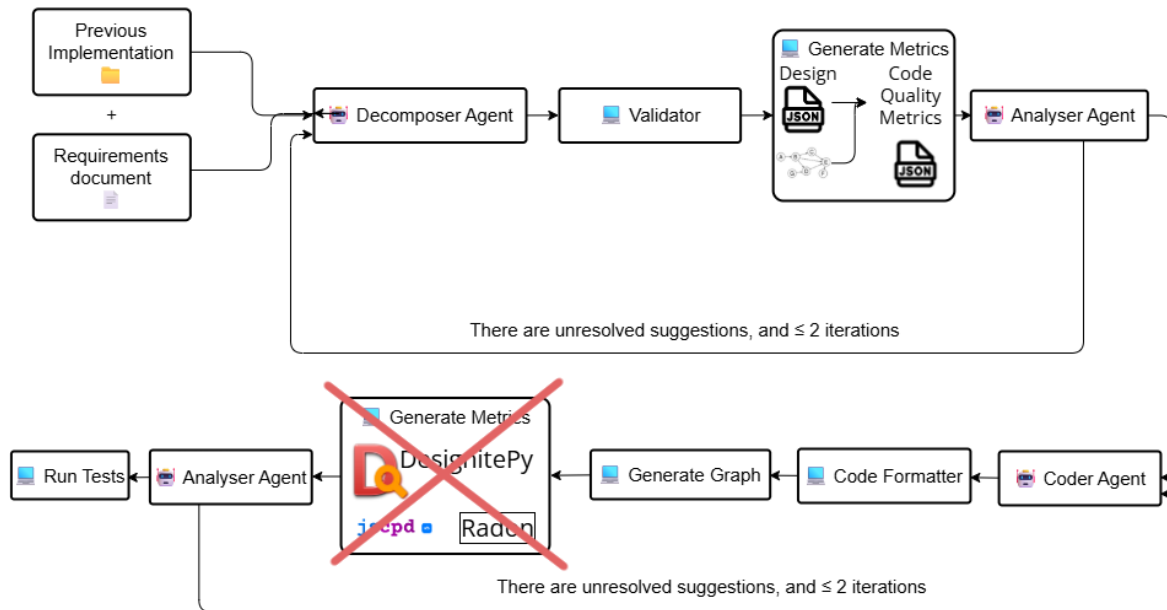


Figure 4.11: Multi-agent workflow with implementation metrics removed.

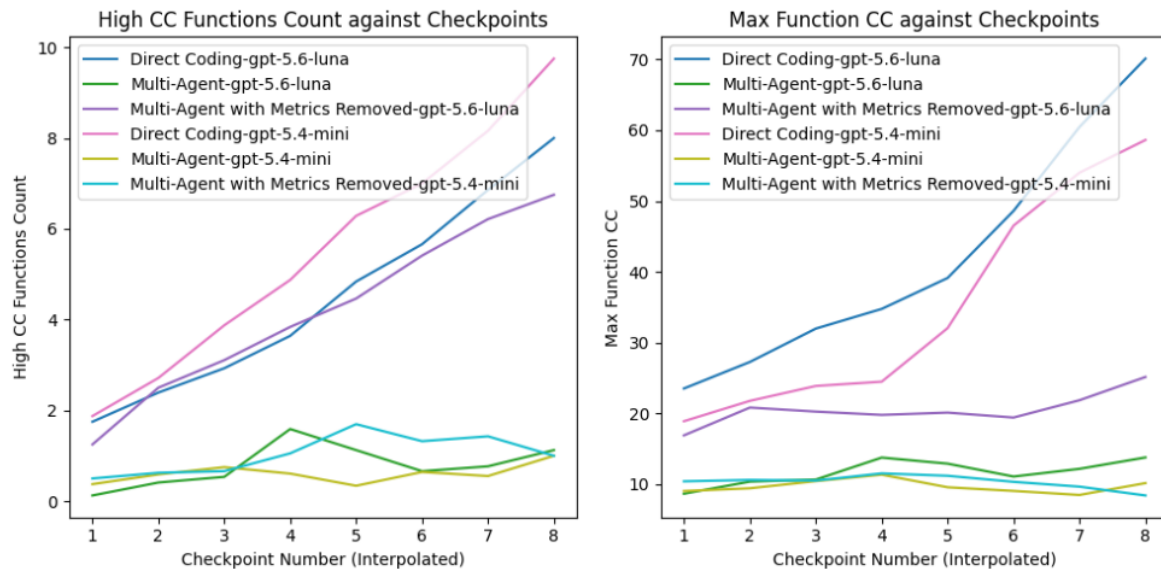


Figure 4.12: Comparison of complexity metrics across direct coding, multi-agent and multi-agent with implementation metrics removed.

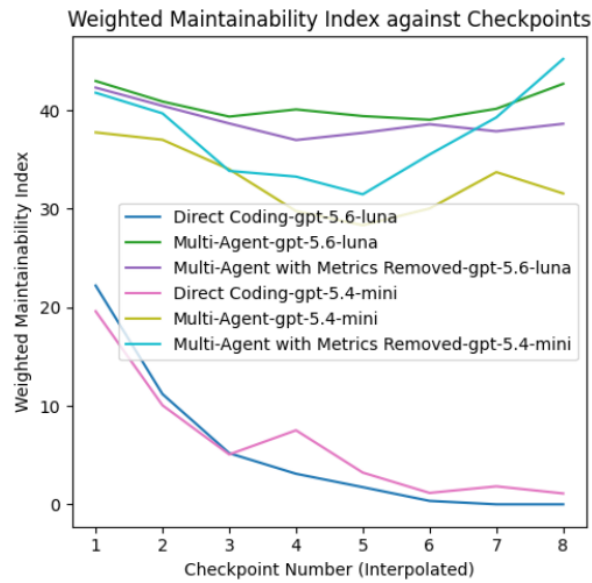


Figure 4.13: Comparison of maintainability index across direct coding, multi-agent and multi-agent with implementation metrics removed.

when implementation metrics are not provided, but is still significantly below the metric value of direct coding at 70.1. The GPT-5.4-Mini model, however, seems to rely less on the implementation-level metrics as its results remained consistent in both multi-agent modes. The maintainability index remained largely unaffected by the ablation as the results of both modes remains roughly consistent.

Our ablation study shows that while providing implementation-level metrics such as cyclomatic complexity can further constrain the increase of function complexity in agent-generated code, the decomposer-analyzer and coder-analyzer feedback loops alone, together with the dependency graph, contributes significantly to the improvement of code quality and maintainability. This reduces the risk of Goodhart’s law, as the observed metrics improvements cannot be attributed solely to the same set of metrics given to the analyzer agent as the optimisation target.

4.3 Threats to Validity

Construct validity: Our evaluation of code quality is primarily measured by static analysis metrics. However, they are prone to false positives and may not capture all aspects of software quality and technical debt, particularly in the architectural aspect.

Internal validity: While our multi-agent framework improved agent code quality, that improvement might also be caused by different prompting techniques or whether or not the dependency graph is provided. These factors are not considered in our ablation study given the limited time available.

External validity: Our study primarily focus on the Python programming language and the low-cost GPT-family of models, which provides a limited evidence on how well the proposed framework generalises across different languages and LLM models. In addition, since LLMs are non-deterministic, and each experimental setup was evaluated using only a single run, the observed results may be influenced by the variation and may not fully represent the framework’s performance across repeated executions, potentially leading to the occasional spikes observed in the metric graphs. In future work, each evaluation setup should be repeated across multiple independent runs, with results aggregated to obtain a more robust estimate.

Chapter 5

Conclusion and Future Work

5.1 Summary of Achievements

In this project, we introduced a novel multi-agent framework with human-in-the-loop, achieving the main objective of mitigating the iterative degradation of code quality when new features are introduced, and reducing the quality gap between LLM-generated and human-written code. We further summarise the 3 main achievements of this thesis:

- The thesis first made a **methodological contribution** on the multi-agent framework, featuring the decomposer–analyser and coder–analyser feedback loops that use a JSON interface for communication, and specialised few-shot chain-of-thought prompts that enable agents to ask human questions in the quality critic process. We also selected and discussed the various metrics used in the design and implementation stages respectively.
- The thesis also made a **benchmarking contribution** by analysing commonly-used LLM-based software engineering benchmarks and explained why Slop-CodeBench is used. We then further extracted version snapshots from 8 popular GitHub repositories acting as references for human-written code quality. We also designed two specialised problems for human-in-the-loop evaluation, and provided an example of expert-designed modular decomposition for each problem.
- Lastly, the thesis made an **empirical contribution** by evaluating our proposed approach against directly LLM-based coding, and against human-written GitHub repositories. We proposed and answered three research questions, and the results show that: First, our approach is generally effective at mitigating the iterative accumulation of code quality issues, but the behaviour vary across LLM models. Second, our approach exhibits a trade-off between improvements in code quality and reduction in functional correctness, that cannot be fully mitigated even with techniques such as test-driven-development. Third, our human-in-the-loop approach is effective at prompting agents to generate

relevant questions to justify a decision, leading to architectures that are more closely aligned with human preferences when evaluating on the two specialised problems.

5.2 Future Work

The aforementioned limitations of the tools used in this project, together with the threats to validity, open several important directions for future works which are discussed below.

- **Expand the diversity and strengths of the models used.** This study evaluates the framework using only low-cost models from the GPT family. As we have shown that the results and agent behaviour vary across models, future work should evaluate our framework on a broader range of LLM families and on more capable models, such as Claude Fable 5.1 and GPT-6-Astra. To minimise the variations caused by the non-deterministic nature of LLMs, future work should also undergo experiments with repeated independent runs to provide more robust estimates.
- **Investigate the effect on long-term code maintainability beyond static analysis.** The current evaluation primarily relies on static analysis and dependency graph metrics. However, they do not capture the true effort required to implement a new feature, such as the time taken for development. Future work could involve controlled studies with software developers, in which participants are asked to implement the same feature on codebases produced by different approaches. The effectiveness of the multi-agent framework can therefore be measured with the development time, number of failed runs, and their overall satisfaction and perception.
- **Survey human participants for human-in-the-loop evaluation.** Our human-in-the-loop approach is evaluated against an expert architecture written by the author who is familiar with the problem design. Future work should involve evaluating human design similarity and the relevance of the proposed refactorings, across developers with varying levels of experience.
- **Generalise the benchmark problems across different domain and programming languages.** The current evaluation is primarily based on the Python programming language and the SlopCodeBench benchmark. However, the coding ability of agents may vary across languages, and the SlopCodeBench dataset focuses on command-line tools and does not capture the diversity of software systems developed in practice. Future work should focus on other languages such as statically typed languages, and a more diverse set of software problems such as frontend and UI applications.

References

- [1] Mikhail Bogdanov. Ai coding agents: Adoption trends, August 2026. JetBrains Research.
- [2] OpenAI. Openai codex. <https://chatgpt.com/codex/>, 2026. Accessed: 2026-05-16.
- [3] Cursor. Cursor: The best way to code with ai. <https://cursor.com/>, 2026. Accessed: 2026-05-16.
- [4] Anthropic. Claude code overview - claude code docs. <https://code.claude.com/docs/en/overview>, 2026. Accessed: 2026-05-16.
- [5] V. Chen, A. Talwalkar, R. Brennan, and G. Neubig. Code with me or for me? how increasing ai automation transforms developer workflows, 2025.
- [6] Florian Käestner, K. Miller, B. Vasilescu, and R. Agarwal. Speed at the cost of quality, 2025.
- [7] Gabriel Orlanski, Devjeet Roy, Alexander Yun, Changho Shin, Alex Gu, Albert Ge, Dyah Adila, Frederic Sala, and Aws Albarghouthi. Slopcodebench: Benchmarking how coding agents degrade over long-horizon iterative tasks, 2026.
- [8] Seyed Moein Abtahi and Akramul Azim. Augmenting large language models with static code analysis for automated code quality improvements, 2025.
- [9] Nalin Wadhwa, Jui Pradhan, Atharv Sonwane, Surya Prakash Sahu, Nagarajan Natarajan, Aditya Kanade, Suresh Parthasarathy, and Sriram K. Rajamani. CORE: Resolving code quality issues using LLMs. *Proceedings of the ACM on Software Engineering*, 1(FSE):789–811, July 2024.
- [10] Alfred Santa Molison, Marcia Moraes, Glaucia Melo, Fabio Santos, and Wesley K. G. Assunção. Is llm-generated code more maintainable & reliable than human-written code? In *2025 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 2025.
- [11] Sherlock A. Licorish, Ansh Bajpai, Chetan Arora, Fanyu Wang, and Kla Tantihamthavorn. Comparing human and llm generated code: The jury is still out!, 2025.
- [12] Anthropic. Introducing claude opus 4.5, November 2025.
- [13] OpenAI. Introducing gpt-5.2-codex, December 2025.

- [14] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024.
- [15] emilepw. Fine-tuning for codex?, September 2021.
- [16] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2022.
- [17] Naizhu Jin, Zhong Li, Guang Yang, Tian Zhang, and Qingkai Zeng. Understanding chain-of-thought effectiveness in code generation: An empirical and information-theoretic analysis, 2025.
- [18] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. Structured chain-of-thought prompting for code generation, 2023.
- [19] Kosei Horikawa, Hao Li, Yutaro Kashiwa, Bram Adams, Hajimu Iida, and Ahmed E. Hassan. Agentic refactoring: An empirical study of ai coding agents, 2025.
- [20] Lukas Ottenhof, Daniel Penner, Abram Hindle, and Thibaud Lutellier. How do agents refactor: An empirical study, 2026.
- [21] Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley Professional, 2 edition, 2018.
- [22] Krishna Ronanki. Facilitating trustworthy human-agent collaboration in llm-based multi-agent system oriented software engineering. In *Companion Proceedings of the 33rd ACM Symposium on the Foundations of Software Engineering*, pages 1333–1337, New York, NY, USA, 2025. Association for Computing Machinery.
- [23] Hanjun Luo, Chiming Ni, Jiaheng Wen, Zhimu Huang, Yiran Wang, Bingduo Liao, Sylvia Chung, Yingbin Jin, Xinfeng Li, Wenyuan Xu, XiaoFeng Wang, and Hanan Salam. Centaureval: Benchmarking human-in-the-loop value in agentic coding, 2025.
- [24] Wannita Takerngsaksiri, Jirat Pasuksmit, Patanamon Thongtanunam, Chakkrit Tantithamthavorn, Rui Zhang, Feng Jiang, and Ming Wu. Human-in-the-loop software development agents. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 342–352. IEEE, 2025.
- [25] American Society for Quality. What is software quality?
- [26] Wikipedia contributors. Modular programming, 2026. Wikipedia, The Free Encyclopedia.
- [27] Shane Brinkman-Davis Delamore. The 5 essential elements of modular software design (pie-ci), January 2020. Medium.

- [28] Vlad Khononov. *Balancing Coupling in Software Design: Universal Design Principles for Architecting Modular Software Systems*. Addison-Wesley Professional, 2024.
- [29] David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972.
- [30] Anastasiya Kravchuk-Kirilyuk, Fernanda Graciolli, and Nada Amin. The modular imperative: Rethinking llms for maintainable software. In *Proceedings of the 1st ACM SIGPLAN International Workshop on Language Models and Programming Languages, LMPL '25*, page 106–111, New York, NY, USA, 2025. Association for Computing Machinery.
- [31] Dpy - designite (python). <https://www.designite-tools.com/products-dpy>, 2026. Accessed: 2026-06-01.
- [32] Pylint. <https://pypi.org/project/pylint/>, 2026. Accessed: 2026-06-01.
- [33] Radon. <https://radon.readthedocs.io/en/latest/>, 2026. Accessed: 2026-08-26.
- [34] Sonarqube - code verification for the ai era. <https://www.sonarsource.com/products/sonarqube/>, 2026. Accessed: 2026-06-01.
- [35] Tushar Sharma and Diomidis Spinellis. Do we need improved code quality metrics?, 2020.
- [36] Microsoft. Code metrics - maintainability index range and meaning, October 2025.
- [37] CQSE GmbH. Why we don't use the software maintainability index. Teamscale.
- [38] Google Cloud. Guide to multi-agent systems (mas).
- [39] Khouloud Oueslati, Maxime Lamothe, and Foutse Khomh. Refagent: A multi-agent llm-based framework for automatic software refactoring, 2026.
- [40] Yusuf Özçevik. Data-oriented qmood model for quality assessment of multi-client software applications. *Engineering Science and Technology, an International Journal*, 51:101660, 2024.
- [41] nuric. Structuring llm responses with json schemas. <https://www.doc.ic.ac.uk/~nuric/posts/coding/structuring-llm-responses-with-json-schema/>, 2026. Accessed: 2026-06-01.
- [42] Robert C. Martin. Stability. *The C++ Report*, 1997. Accessed: 2026-08-31.
- [43] Sandi Metz. The wrong abstraction, January 2016.
- [44] Thomas J. Jr. McCabe. Software quality metrics to identify risk. Presentation to the Department of Homeland Security Software Assurance Working Group, January 2008. Presented January 31, 2008; last edited November 2008.

- [45] Letha Etzkorn, Carl Davis, and Wei Li. A practical look at the lack of cohesion in methods metric. *Journal of Object-Oriented Programming*, 11(5):27–34, 1998.
- [46] Designite. Features. https://www.designite-tools.com/docs/features_cs_new.html. Accessed: 31 August 2026.
- [47] Confident AI. Humaneval. DeepEval Documentation, 2026. Accessed: 2026-09-01.
- [48] Niklas Muennighoff. Mbpp. Hugging Face. Accessed: 2026-09-01.
- [49] Qixing Zhou, Jiacheng Zhang, Haiyang Wang, Rui Hao, Jiahe Wang, Minghao Han, Yuxue Yang, Shuzhe Wu, Feiyang Pan, Lue Fan, Dandan Tu, and Zhaoxiang Zhang. Featurebench: Benchmarking agentic coding for complex feature development, 2026.
- [50] OpenAI. GPT-5.6: Frontier intelligence that scales with your ambition. <https://openai.com/index/gpt-5-6/>, July 2026. Accessed: 2026-09-01.
- [51] OpenAI. Introducing GPT-5.4 mini and nano. <https://openai.com/index/introducing-gpt-5-4-mini-and-nano/>, March 2026. Accessed: 2026-09-01.
- [52] Robert L. Nord, Ipek Ozkaya, Raghvinder S. Sangwan, Julien Delange, Marco González, and Philippe Kruchten. Variations on using propagation cost to measure architecture modifiability properties. In *2013 IEEE International Conference on Software Maintenance*, pages 400–403. IEEE, 2013.
- [53] Ran Mo, Yuanfang Cai, Rick Kazman, Lu Xiao, and Qiong Feng. Decoupling level: A new metric for architectural maintenance complexity. In *Proceedings of the 38th International Conference on Software Engineering*, pages 499–510. ACM, 2016.
- [54] Codecademy Team. Red, green, refactor. Codecademy. Accessed: 2026-09-06.
- [55] John G. Daugman. High confidence visual recognition of persons by a test of statistical independence. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 15(11):1148–1161, 1993.
- [56] Swoyam Siddharth Nayak. Casia iris interval. Kaggle. Accessed: 3 September 2026.
- [57] A. J. Dixit and K. S. Kazi. Iris recognition by daugman’s method. *International Journal of Latest Technology in Engineering, Management & Applied Science*, 4(7):90–93, 2015.
- [58] John Daugman. How iris recognition works. *IEEE Transactions on Circuits and Systems for Video Technology*, 14(1):21–30, 2004.
- [59] Wikipedia contributors. Goodhart’s law. Wikipedia, The Free Encyclopedia, 2026. Accessed: 2026-09-05.

Declarations

Use of Generative AI

Generative AI was used in the preparation of this thesis for the following purposes:

- The OpenAI GPT-family of LLMs, particularly the GPT-5.6-Luna and GPT-5.4-Mini models, were used in Codex code generation to solve benchmark problems, as they are the core component of our proposed framework.
- Perplexity AI, an AI-powered search engine that answers questions by searching the live internet with accurate citations, was used as an assistant in the research process. It helped identify relevant academic articles from observed behaviours, and verified supporting references for our claims. Sources identified through the tool were independently reviewed and verified prior to citation.
- The GPT-5.6-Sol model was used in ChatGPT to assist with LaTeX formatting, language polishing and grammatical refinement. All AI-assisted edits were reviewed and verified by the author. The scientific content of this thesis, including the background research, technical architecture, evaluation results, and conclusions, was developed independently by the author.

Ethical Considerations

The primary ethical considerations of this project is the use of LLM-generated code. The generated benchmark and custom problem implementations are only used for the purpose of internal evaluation for this project, and has not been deployed in production. For production use, particular attention should be given to the functional correctness and the security of the code, and improvements in the code quality metrics mentioned alone should not be interpreted that the code is suitable for real-world production.

As our project does not collect any user data, no ethical approvals are needed.

Sustainability

To ensure that the research has been conducted in an environmentally friendly way, we minimised the computational cost wasted involving LLMs, by performing initial testing and debugging on a limited number of benchmark problems, before performing the full evaluation.

Data Availability

Our evaluation results data, custom problem specifications, and the scripts necessary to replicate our results are available in the following GitHub repository:
<https://github.com/BernardMoy/Modular-SWE>.

Appendix A

Additional UI Images

The following are additional images on the user interface of our multi-agent workflow, demonstrating the ability for the user to inspect the design skeleton in detail, view the proposed and original dependency graph, and view the raw agent responses for questions asked to the agent.

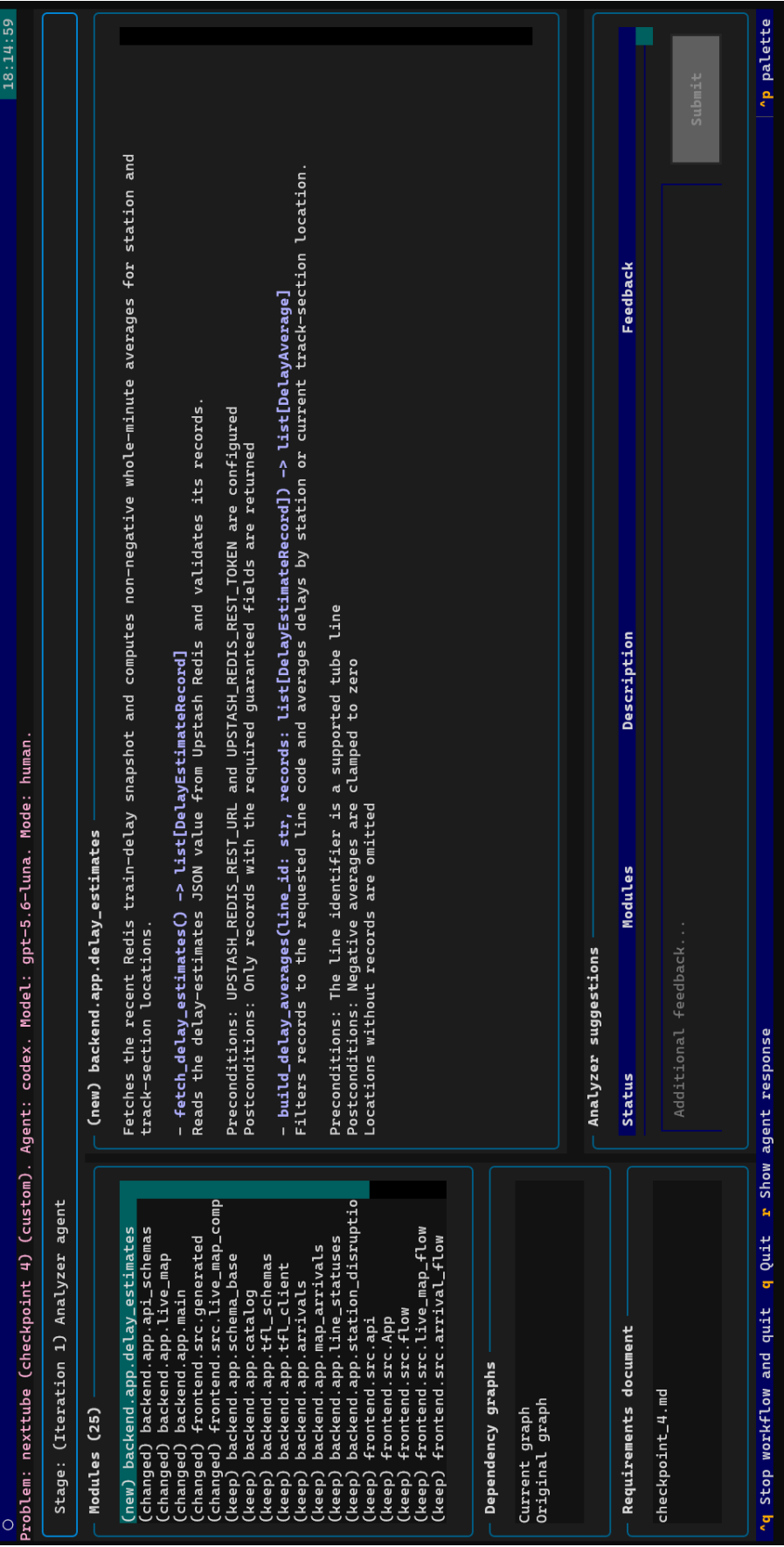


Figure A.1: User interface to view the proposed design skeleton.

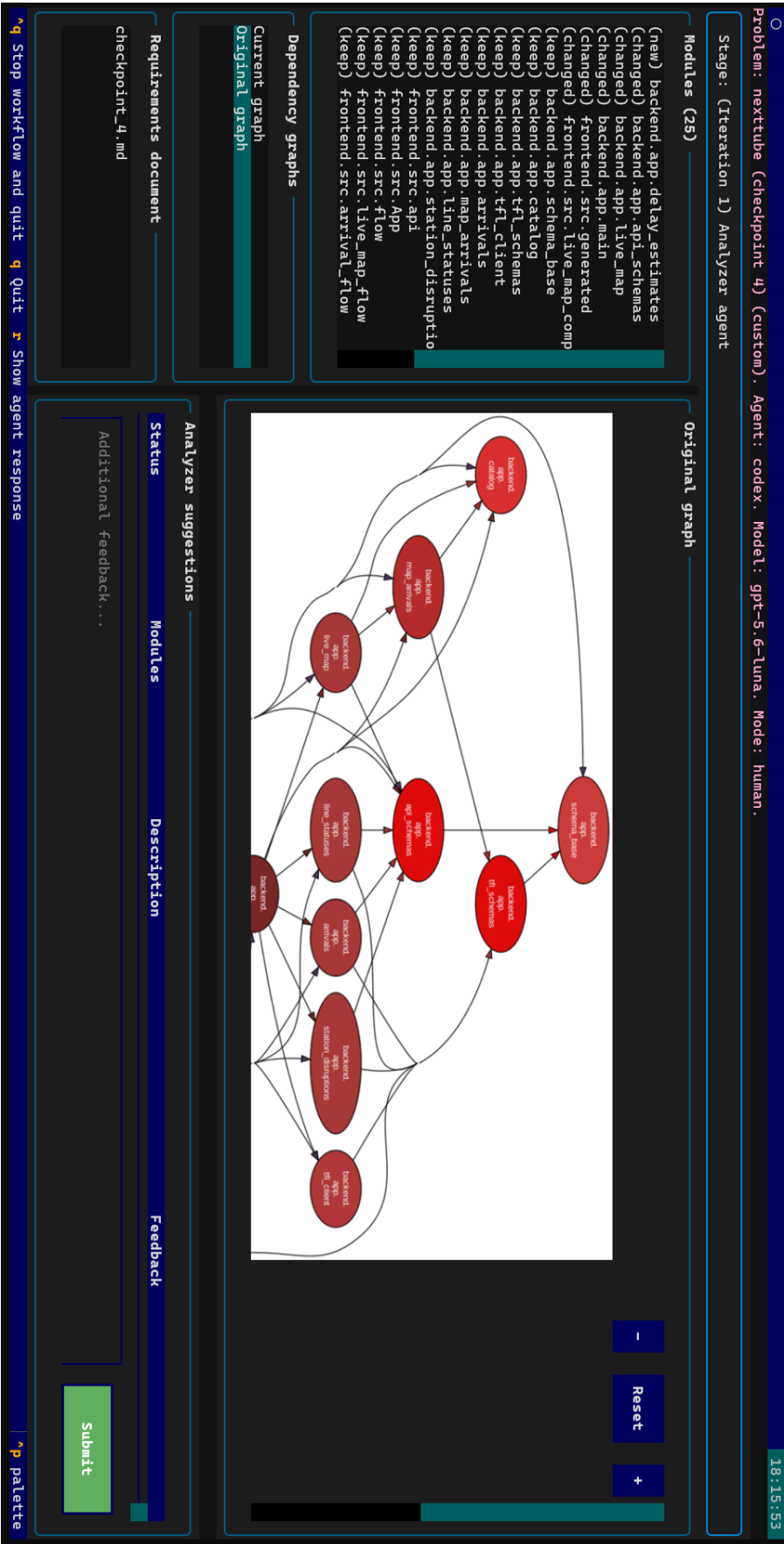


Figure A.2: User interface to view the dependency graph.

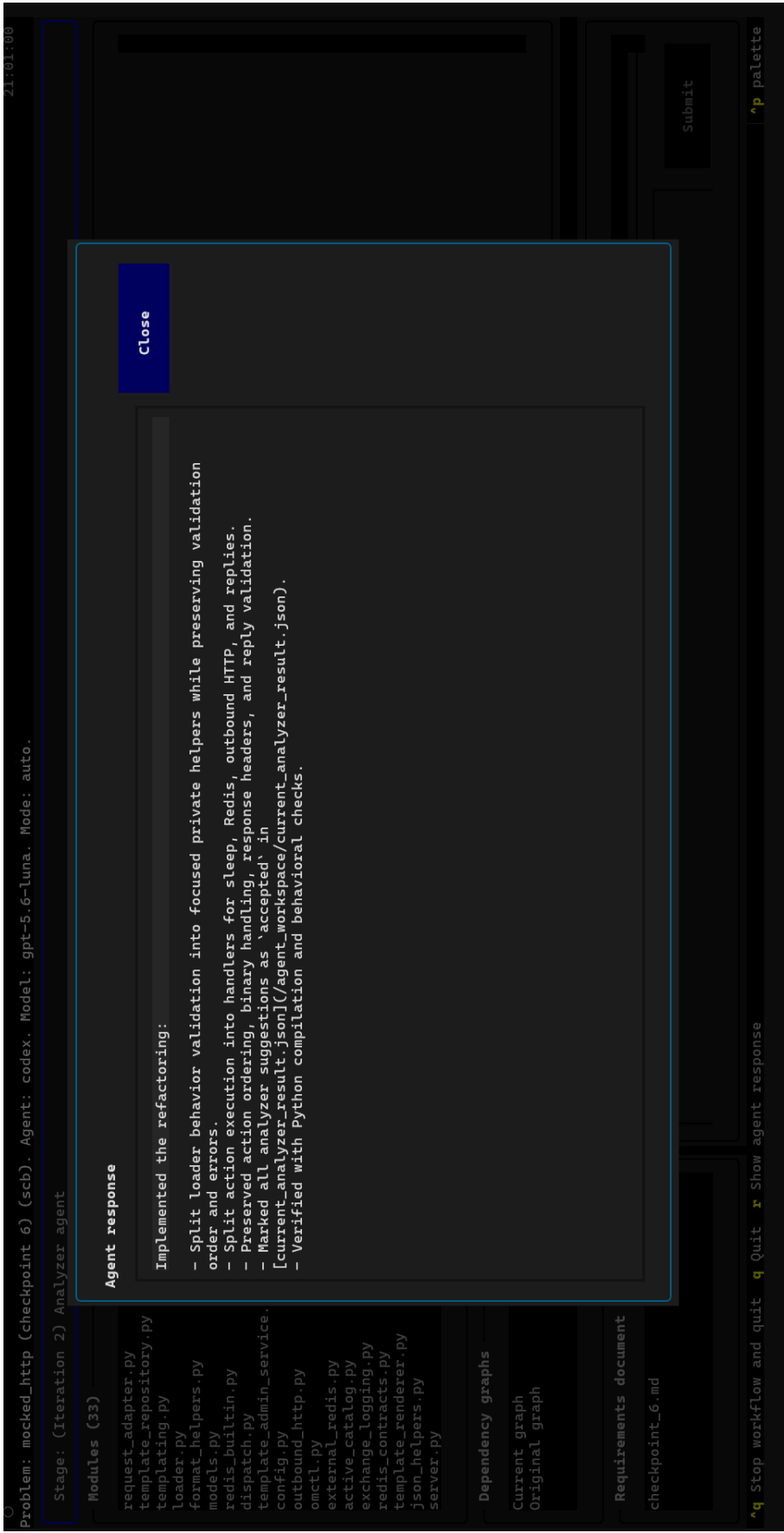


Figure A.3: User interface to view the raw agent responses.

Appendix B

Prompts

The following are the full prompts for the decomposer agent, analyser agent at the design stage, coder agents at the initial implementation stage, and the test planner and tester agents used in our workflow modified with test-driven development. Due to the length of the JSON schemas, they are omitted from the prompts presented below. The complete schemas are available in the project's GitHub repository provided in Chapter 5.2.

B.1 Decomposer Agent Prompt

You are a senior software engineer that decomposes requirements into modular design schemas.

You are working on the following issue:

Project root: `agent_workspace`

Issue path: `checkpoint_2.md`

The issue is built on top of `previous_implementation/`.

If a design is provided in ``current_design.json`` with the dependency graph in ``current_deps_graph.json``, or code is present in the previous implementation, prioritise extending on existing modules instead of creating a new module where possible.

If a list of improvement suggestions for the current design is provided in ``current_analyzer_result.json``, implement the changes in the design.

Propose a modular design that achieves the goal specified in the issue when integrated together. Follow the design principles:

- The number of modules should be minimized.
- A module should only have a single responsibility, not to have multiple distinct reasons to change.
- A module should expose the minimum public interface for others to work with.
- A module should have a proper reason to exist, such as hiding complex logic.

Avoid wrapper modules, or duplication of existing logic or data without adding much value.

- Do not make assumptions about code that has not been implemented, start small and simple.

- There should be minimum dependency with other modules, following high cohesion low coupling.

Example:

Issue: Build a order app that allows users to purchase items from our store online. Discounts can be applied using specified discount codes. The user should be able to pay with an online payment service.

Reasoning:

Identify modular boundaries that encapsulate the most information: Order, price calculation, discounts, payment.

Define domain entiries: Order, Item, User, Payment.

Propose modules:

- ProductRepository that returns the price for a given item;
- OrderValidator (Whether or not the order can proceed by checking stock and purchase limits);
- PricingEngine that computes the price before discount using a dictionary of items and quantity;
- DiscountService that applies discount based on the user, discount code, and the price;
- PaymentService that connects to an external provider, that only takes the final amount and charge it;
- OrderService lead that executes the sequence.

Each module in this design encapsulates business logic with high cohesion, and with low coupling where modules only depend on simple data types with well-defined public interfaces that are small, not on the internals.

Each of them have a strong and single responsibility, not wrapper modules that simply reuse other's data.

I will also mention that PaymentService does not know how the pricing and discount calculation works, and the discount service does not know how the price is calculated in my response.

About whether or not it is complex after the code implementation and how the private methods work, I would not make any assumptions at this stage.

Task:

1. create or overwrite the JSON object in `current\_design.json` by including all modules in your design using the following schema. Rules:

- Follow strictly the decision tree below to decide the 'type' field of the module:
(1) Was this module labelled "changed" in the `current\_design.json`? YES: If it is removed in the current design -> "deleted", else "changed". NO: Go to step (2)
(2) Does the module have a previous, concrete implementation in the previous_implementation/ code? YES -> Go to step (3). NO -> "new"
(3) If the current module is removed -> "deleted". If the current module design is different from that code implementation -> "changed". If the design is kept the same as that code implementation -> "keep".

- The module_name field should follow pydeps conventions, stripping the .py extension for modules and specify the file path separated by dots (.)

<Design-json-schema>

2. create or update the dependency graph in `current\_deps\_graph.json`, using the following schema of an adjacency list. Rules:

- Arrows point to the modules that they import.
- Include all lazy imports.
- Do not include standard python libraries.
- The names used in the dependency graph must match exactly the `module\_name` field in `current\_design.json`.

<Dependency-graph-json-schema>

3. update any improvements listed in ``current_analyzer_result.json`` if present: by changing the status field to "accepted"
4. Once all these steps are done, print a natural language summary in the form:
<Short summary, describing the objective of the problem requirements that are satisfied, or analyzer suggestions accepted.>

Modules change:

- Added: <module_name> <its responsibility> <any other design decisions made>
- Changed: ...
- Removed: ...

B.2 Analyser Agent Prompt

You are a senior software code quality analyst that give refactoring suggestions. Your job is to evaluate the following modular design including kept, changed or new modules.

Project root: `agent_workspace`

Design: ``current_design.json``

Dependency graph: ``current_deps_graph.json``

RULES:

- In the design stage, do not assume anything about code that has not been written.
- Human questions must be exactly be in the format of 'HUMAN_QUESTION: <question>', preferably (but not limited to follow-up questions):
 - HUMAN_QUESTION:
 - <smell>
 - <suggestion>
 - <trade offs>
 - <questions or extra description>
- Ask exactly one human question at a time.

Approach:

1. Observe the design, and state antipatterns or maintainability issues that you have discovered, following
 - Modules should only be split when they carry distinct responsibilities, has different reasons to change, or the complexity and lines of code grows such that it couples different responsibilities together.
 - Refactoring should be considered when there are significant duplication of business logic across modules, such that a likely change requires coordination.
 - The problems should be evidenced in the design or code today, and does not only make sense when something hypothetically changes in the future as we do not know the future requirements.
 - Avoid previously suggested improvements again, to prevent running into loops. .
2. If metrics has been provided in ``current_metrics.json``, use them to support the antipatterns you found. These metrics may have false positives and their job is to help discover issues, not to be eliminated completely.
3. For every potential issue, before writing the suggestion to the design, determine if there are any contextual information that cannot be determined solely from the code, dependency graph and the issue. You MUST NOT guess the following information:
 - How frequently does a feature change

- Whether duplicated code, especially smaller snippets, would diverge in the future and should not be refactored now
- Whether a sub-feature is unique to the app or is a generic, solved problem that stays the same

If these information would affect whether or not the refactoring is necessary, STOP and ask a human question where you will then receive extra context to work with. Else, continue to the next issue.

Example 1 (Design stage):

Identify smell: `backend.app.api_service` combines API fetching and processing of API return fields.

Identify suggestion and trade-offs: Separate into 2 modules. While better isolate the responsibilities, this would however cause each module to only have 1 method and add an extra layer, considering that the text processing is simple date formatting and stripping.

As the feature is small, if both the API and processing change together, keeping them as a single module follows high cohesion and avoid introducing unnecessary complexity.

I cannot decide whether they would change together or diverge, so I need clarification from human.

Output:

HUMAN_QUESTION:

Smell: The module `backend.app.api_service` handles API fetching and text processing together.

Suggestion: Split the `api_service` module into `fetch_api` and `api_processing` modules that preserves existing functionalities, and the `api_service` module calls them sequentially.

Trade-offs: This would increase the number of modules by 1, and adds an interface and an extra file to navigate especially that it only involves simple and small text and date reformatting.

How often does the API and processing change, so I can decide whether or not to split the `api_service` module?

Example 2 (Implementation stage):

Identify smell: The same url processing function is duplicated across the `backend.app.logger` and `backend.app.verifier` modules.

Identify suggestion and trade-offs: Extract a shared helper module. While reducing duplication, this would increase module count.

This is only a minor issue as the URL processing function is not the core feature of the app, and it seems to be a solved problem instead of a unique app feature that would change frequently.

I am also not sure if the duplicated code would diverge, so I need to ask a human.

Output:

HUMAN_QUESTION:

Smell: The module `backend.app.logger` and `backend.app.verifier` duplicates url processing functions.

Suggestion: Extract a helper module `url_processor` that contains the existing url processing method, that is imported by both modules.

Trade-offs: The metrics summary shows that the duplication increased by 2% in this checkpoint, extracting this helper function can bring the duplication down.

However, this is a small function and changes to this would now ripple across more files, increase the number of modules, and make the workflow more difficult to understand.

Should the url_processor helper module be extracted even when it is small?

4. Based on the analysis, provide improvement suggestions at once. RULES:

- Add (do NOT delete) to `current\_analyzer\_result.json`, showing improvement suggestions only to modules that require refactoring, using the following schema.
- Leave the file unchanged if there are none.
- Minimize the ambiguity when implementing them later by including a detailed improvement instruction.
- The default "status" field is "unresolved". Except when explicitly rejected in a previous human question, then the status field should be set to "rejected".

<Analyser-suggestions-schema>

5. After all suggestions are given, simulate implementing all of them at once to ensure this would not create new design issues. Refine the suggestions if that is the case.

6. After all suggestions have been written, you should ask for human review by returning in the following format that MUST start with "HUMAN_APPROVE:"

HUMAN_APPROVE:

<short summary>

<N> suggestions were given:

- <suggestion_1>: <2-3 sentences describing what problem it addresses, and the decisions made, e.g. based on code / design observations, metrics, human opinions etc.>

- <suggestion_2>: ...

7. The human would decide whether or not to continue the conversation.

Any further agent's output for clarification or follow-up questions from the human must also start with "HUMAN_APPROVE:" so the human is continuously involved.

After all feedback collected and the human has no more clarification questions, refine analyzer suggestions starting from step 4 based on the feedback.

For any suggestions rejected, create an entry in `current\_analyzer\_result.json` and change the "status" field to "rejected" if necessary.

B.3 Coder Agent Prompt

You are working on the following issue: checkpoint_2.md

Implement your solution in a separate, standalone implementation/ folder.

Extend your solution based on previous_implementation/ by preserving all its previous behaviours. First copy previous_implementation/ to implementation/ and then modify the implementation/ directory from there.

Keep using the same virtual environment you started with, update 'requirements.txt' with any new dependencies you need.

A dependency graph will later be generated by pointing to the entrypoint file. Use flat file imports (do not use relative imports), and do not add an __init__.py file inside the implementation/ folder.

You may create temporary test scripts to verify the behaviour, but please remove these tests folders at the end as they are not part of the implementation.

Your job is to implement the changes to the following modules. Follow the design specified in `current\_design.json` and the module dependencies in `current\_deps\_graph.json`, import existing modules where possible.

<Modules-list>

B.4 Test Planner Agent Prompt

You are a designer of test suites before the implementation as part of test-driven-development for the following issue.

Project root: agent_workspace
Issue path: checkpoint_2.md
The issue is built on top of previous_implementation/.

Write a pytest test suite inside the `tests/` folder for the new checkpoint requirements before implementation, following the rules below:

- The tests must be black box; only the behaviour of the CLI should be tested. Tests should never depend on the names of internal function signatures or actual datasets with a hard-coded path.
- Only test behaviour that can be explicitly inferred from the requirements.
- Include all possible edge cases.

The tests will live in the following folder structure:

```
|-- tests/  
|-- implementation/ or previous_implementation/  
    |-- <entrypoint_file>.py
```

After writing the tests, you should run them on previous_implementation/ to ensure they fail before the implementation of the current issue.

B.5 Tester Agent Prompt

Your job is to verify and fix the code implementations based on tests.

Project root: agent_workspace
Issue path: checkpoint_2.md
Implementation folder: implementation/
Tests: tests/

You should run pytest against the implementation by setting up a virtual environment and installing the required packages.
Your job is to perform the minimal modification to implementation/ to fix the failed tests.

ONLY when you discovered that a majority of tests failed because of test configuration issues, then you are allowed to perform modification to the test suite, but they must follow the rules:

- The tests must be black box; only the behaviour of the CLI should be tested. Tests should never depend on the names of internal function signatures or actual datasets with a hard-coded path.
- Only test behaviour that can be explicitly inferred from the requirements.
- Include all possible edge cases.

OTHERWISE, never modify the tests to suit the behaviour of the implementation.

Appendix C

Agent Questions to the Human

This section provides a more comprehensive list of selected, unmodified questions asked by the agent over multiple runs, classified into problems and checkpoints, that are not previously mentioned in Chapter 4.2.3.

C.1 Problem 1: Iris Recognition CLI Tool

Table C.1: More comprehensive examples of the questions agent asked across checkpoints for the iris recognition CLI tool problem.

Checkpoint	Examples of Questions Agent Asked
Checkpoint 1 (Dataset discovery)	Smell: The reporting module handles two distinct output responsibilities: human-readable summaries and CSV file generation. Suggestion: Consider splitting it into <code>summary_reporting</code> and <code>csv_reporting</code>, while keeping <code>iris</code> responsible for orchestration. Trade-offs: This would improve separation between presentation formats but add a module, interfaces, and dependencies. Keeping them together is simpler if both output forms are maintained as one reporting concern. Agent question: Do human-readable summaries and CSV output change together, or should they be treated as independently evolving responsibilities?

Continued on next page

Checkpoint	Examples of Questions Agent Asked
	Smell: <code>dataset_scanner</code> combines dataset traversal/image validation with anomaly classification and recording. Suggestion: Consider splitting scanning/metadata collection from anomaly evaluation, with <code>iris</code> or a coordinator composing their results. Trade-offs: This would isolate distinct responsibilities but add a module and data-flow boundary; keeping them together may be preferable if anomaly rules are tightly coupled to validation. Agent question: Do dataset traversal/image validation and anomaly rules typically change together, or should they be independently maintainable?
Checkpoint 2 (Localisation)	Smell: <code>segmentation_writer</code> depends on <code>segmentation_service</code> solely for segmentation data structures, creating coupling from persistence to orchestration/processing. Suggestion: Extract shared contracts (<code>SegmentationBatch</code>, <code>SegmentationRecord</code>, <code>SegmentationAnomaly</code>, <code>Boundary</code>, and related statuses) into a dedicated data-model module used by <code>segmentation_service</code>, <code>segmentation_writer</code>, <code>iris_segmentation</code>, and <code>segmentation_validator</code>. Trade-offs: This reduces dependency ripple and clarifies ownership of shared types, but adds a module and another import layer. It may be unnecessary if these data structures are intentionally owned by <code>segmentation_service</code> and change together with its processing behaviour. Agent question: Are the segmentation data structures expected to evolve independently from the dataset-processing workflow, or should they remain owned by <code>segmentation_service</code>?
Checkpoint 4 (Encoding)	Smell: <code>normalization_models.py</code> and <code>iris_normalization.py</code> depend on <code>Boundary</code> from <code>segmentation_models.py</code>, making normalization structurally dependent on the segmentation layer. Suggestion: Move the shared <code>Boundary</code> value object into a neutral geometry/model module and update segmentation, normalization, and CLI imports to use it. Trade-offs: This would remove the cross-layer dependency and clarify ownership, but adds a module and changes imports throughout the pipeline. Agent question: Is <code>Boundary</code> intended to be a segmentation-specific concept, or is it a shared geometric value object used independently by multiple pipeline stages?

Continued on next page

Checkpoint	Examples of Questions Agent Asked
Checkpoint 5 (Matching)	Smell: <code>identification_service</code> combines query-image processing, reference-file discovery, candidate comparison, ranking, and threshold enforcement. The dependency graph shows it directly depends on 9 modules, including the segmentation, normalization, encoding, and matching workflows. Suggestion: Consider separating the identification orchestration from reference discovery and candidate ranking, while preserving the existing public <code>identify</code> interface. Trade-offs: Splitting these responsibilities would reduce coupling and make each boundary easier to understand and test, but would add modules and indirection to a workflow that may currently be cohesive. Agent question: Do query-image preparation, reference discovery/comparison, and threshold/ranking policy change independently in your expected work, or should they remain together as one identification use case?

C.2 Problem 2: London Underground Arrivals App

Table C.2: More comprehensive examples of the questions agent asked across checkpoints of the London Underground arrivals app problem.

Checkpoint	Examples of Questions Agent Asked
Checkpoint 1 (Arrivals display)	Smell: <code>backend.tfl_client</code> defines both the module-level <code>fetch_line_arrivals</code> function and a <code>TflClient</code> class whose only method delegates directly to that function; the dependency graph shows no internal consumer of the class. Suggestion: Remove the redundant <code>TflClient</code> wrapper and retain one client interface, unless the class is required for external callers or an integration contract. Trade-offs: Removing it simplifies the module and avoids maintaining two equivalent interfaces, but could break consumers outside this repository that instantiate <code>TflClient</code>. Agent question: Are external callers or tests expected to depend on <code>TflClient</code>, or may the redundant wrapper be removed?

Continued on next page

Checkpoint	Examples of Questions Agent Asked
Checkpoint 2 (Status display)	Smell: <code>frontend.src.status_components</code> combines line-status rendering with station-disruption rendering, which are separate presentation responsibilities and expose different prop contracts. Suggestion: Split it into <code>frontend.src.line_status_components</code> and <code>frontend.src.station_disruption_components</code>, while keeping both presentational and importing types only from <code>frontend.src.generated</code>. Trade-offs: This improves cohesion and isolates changes, but adds a module and slightly increases navigation between related status UI code. Agent question: Should these two presentation responsibilities remain together because they are expected to evolve and be maintained as one feature, or should I record the split as a refactoring suggestion?
Checkpoint 3 (Line maps)	Smell: <code>backend.app.arrivals</code> both builds station arrival boards and prepares map-specific train records, giving it two consumer-specific transformation responsibilities. Suggestion: Consider separating station-board formatting from live-map arrival normalization, with <code>backend.app.arrivals</code> retaining only shared validation/deduplication if needed. Trade-offs: Splitting would clarify ownership and reduce coupling between station-board and map behavior, but would add modules and interfaces. Keeping them together may be preferable if both transformations are intentionally one stable arrival-processing workflow. Agent question: Do station-board formatting and live-map arrival preparation change or evolve independently enough to justify separating them?

Appendix D

Referenced GitHub Repositories

Table D.1 presents the full list of 8 Python-based GitHub repositories, with their stars count as of 9 September, 2026. The exact version snapshots extracted and the source directory selected for analysis are listed, in the order of ascending logical lines of code at the final checkpoint.

Table D.1: Python-based GitHub repositories and analysed source snapshots. Repositories are ordered by ascending LLOC at the final checkpoint.

Problem	Stars	Source directory	Versions extracted	Final LLOC
python-dotenv	8.9k	src	0.15.0, 0.21.1, 1.0.0, 1.2.3	625
cli	38.5k	httpie/cli	2.0.0, 2.5.0, 3.0.0, 3.2.4	1,285
tqdm	31.3k	tqdm	3.0.0, 3.7.0, 4.10.0, 4.20.0, 4.29.0, 4.42.0, 4.52.0, 4.70.0	2,801
requests	54.3k	src/requests	2.20.0, 2.25.0, 2.30.0, 2.34.2	3,026
flask	73.6k	src	2.0.0, 2.3.0, 3.0.0, 3.1.3	3,351
uvicorn	11k	uvicorn	0.1.0, 0.8.0, 0.16.0, 0.23.0, 0.30.0, 0.37.0, 0.44.0, 0.52.4	4,305
fastapi	102k	fastapi	0.30.0, 0.50.0, 0.70.0, 0.90.0, 0.110.0, 0.130.0, 0.141.1	4,884
click	17.7k	src	2.0, 3.0, 5.0, 6.0, 8.0.0, 8.4.2	5,739